



北京航空航天大学
BEIHANG UNIVERSITY



数据结构与程序设计（信息类）

Data Structure & Programming

北京航空航天大学 数据结构课程组

软件学院 谭火彬

2024年春



提纲：数据结构程序设计基础（C语言）

- ◆ 1.1 字符串
- ◆ 1.2 二维（多维）数组
- ◆ 1.3 指针
- ◆ 1.4 结构和联合
- ◆ 1.5 文件处理



提纲：数据结构程序设计基础（C语言）

◆ 1.1 字符串

✓ 1.1.1 字符串定义和操作

✓ 1.1.2 字符串查找（匹配）

◆ 1.2 二维（多维）数组

◆ 1.3 指针

◆ 1.4 结构和联合

◆ 1.5 文件处理



字符串 (string)

- ◆字符串是一种常见的数据类型：一组字符排列
 - ✓单词、语句、段落、文章
- ◆C语言没有提供相应的基本数据类型，需设计一种**数据结构**来实现该数据类型

串的定义

串是由 $n \geq 0$ 个字符组成的有限序列，记为

$$S = "a_1 a_2 a_3 \dots a_{n-1} a_n"$$

其中：S表示串名（也称串变量），一对引号括起来的字符序列称为串值
 a_i 可以是字母、数字或其他允许的字符
 n 为串的长度，长度为0的串称为空串



串的基本操作（算法）

C函数

1. 给串变量赋值 ASSIGN(S1,S2)
2. 判断两个串是否相等 EQUAL(S1,S2)
3. 两个字符串连接 CONCAT(S1,S2)
4. 求字符串的长度 LEN(S)
5. 求子串 SUBSTR(S,i,k)
6. 求子串在主串中的位置 INDEX(S1,S2)
7. 串的替换 REPLACE(S,S1,S2)
8. 串的复制 COPY(S1,S2)
9. 串的插入 INSERTS(S1,i,S2)
10. 串的删除 DELETES(S,i,k)

.....

strcpy(S1,S2)

strcmp(S1,S2)

strcat(S1,S2)

strlen(S)

strstr(S1,S2)

strcpy(S2,S1)



C语言中的字符串实现

$$S = "a_1 a_2 a_3 \dots a_{n-1} a_n"$$

两种实现方式:

1. 字符数组: `char s[N];`

2. 字符指针: `char *s;` //注意: 需要分配存储空间

0	1	2	...	...	n-2	n-1	n		N-1
a ₁	a ₂	a ₃	...	...	a _{n-1}	a _n	\0	...	...

由于每个字符串的长度各不相同, 如何表示字符串的结束?

1. 记录当前字符串的长度 (普通数组中常用的方法)

缺点: 需要单独的变量存储长度, 管理和使用比较麻烦

2. 在有效字符串的最后添加一个特殊字符: `'\0'`

缺点: 浪费一个存储空间



C语言中的字符串处理函数

- ◆ C语言提供了一系列字符串处理功能函数，所有这些函数只能处理字符串（即一定要有**\0结尾**）
- ◆ `<stdio.h>`，提供各类字符串输入输出转换函数
 - ✓ `scanf`、`printf`中提供%s操作字符串
 - ✓ `puts`、`gets`字符串输入和输出函数
 - ✓ `sscanf`、`sprintf`提供字符串和普通数据类型的转换
- ◆ `<string.h>`，以str开头，提供各类操作函数
 - ✓ 字符串长度：`strlen`
 - ✓ 字符串拷贝：`strcpy`、`strncpy`
 - ✓ 字符串连接：`strcat`、`strncat`
 - ✓ 字符串比较：`strcmp`、`strncmp`
 - ✓ 字符串查找：`strstr`、`strchr`、`strrchr`



C语言中的字符处理函数

- ◆ 字符串基本构成单位为字符，字符处理函数可以简化字符处理
- ◆ `<ctype.h>`：字符处理函数
- ◆ 判断各类字符类型
 - ✓ 判断字母、数字等：isalpha、islower、isupper、isdigit、isalnum
 - ✓ 判断特殊字符：isprint、isspace、isxdigit、iscntrl、ispunct、isgraph
- ◆ 字符大小写转换
 - ✓ 将字符c转换为小写字母：tolower
 - ✓ 将字符c转换为大写字母：toupper



示例：C语言中字符串的定义和赋值

```
//定义一个长度为32的字符数组，可以最大长度为31的字符串
char arr1[32];
//定义一个字符指针，需要指向某个有效的连续存储空间，该空间内容可看做字符串
char *str1;

strcpy(arr1, "Data Structure");
str1 = (char *) malloc(strlen(arr1) + 1); //根据arr1的大小分配字符串空间
strcpy(str1, arr1);
free(str1); //malloc分配的空间，必须通过free释放

//定义一个字符数组，初始化数组成员
char arr2[] = "Data Structure";
//定义一个字符指针，指向某个字符串常量
char *ptr2 = "Data Structure";
```



D01-01: 将数字字符串转换为整数

"123" > 123

"123"

$12 * 10 + '3' - '0' = 123$

$1 * 10 + '2' - '0' = 12$

$'1' - '0' = 1$

$n = 0;$

for($i=0$; $s[i]$ 为数字字符; $i++$)

$n = 10 * n + s[i] - '0';$



问题1：代码实现

```
int atoi(char s[])
{
    int i, n, sign;
    for (i = 0; s[i] == ' ' || s[i] == '\n' || s[i] == '\t'; i++)
        ; /* skip white space */
    sign = 1;
    if (s[i] == '+' || s[i] == '-')
        sign = (s[i++] == '+') ? 1 : -1;
    for (n = 0; s[i] >= '0' && s[i] <= '9'; i++)
        n = 10 * n + s[i] - '0';
    return (sign * n);
}
```

空语句

```
#include <stdio.h>
int atoi(char s[]);
int main()
{
    char s[20];
    scanf("%s", s);
    printf("%d\n", atoi(s));
    return 0;
}
```



D01-02: 将整数转换为字符串

```
void itoa(int n, char s[])
{
    int i, sign;
    if ((sign = n) < 0)
        n = -n;
    i = 0;
    do
    {
        s[i++] = n % 10 + '0';
    } while ((n /= 10) > 0);
    if (sign < 0)
        s[i++] = '-';
    s[i] = '\0';
    reverse(s); //将字符串颠倒
}
```

```
#include <stdio.h>
void itoa(int n, char s[]);
int main()
{
    int n;
    char s[20];
    scanf("%d", &n);
    itoa(n, s);
    printf("%s\n", s);
    return 0;
}
```

提示：关于字符串和其他数据类型的转换，推荐使用**sscanf**和**sprintf**这两个函数



使用字符串转换函数：sprintf

◆将多个普通变量输出为sprintf：变量->字符串

- ✓printf将不同变量输出到屏幕上，屏幕输出的本质上就是字符串
- ✓sprintf用法与printf一样，只不过不是输出到屏幕，而是直接保存到字符串中
- ✓比printf多一个参数（第一个参数），就是要存储的字符串数组，后面参数的使用与printf完全

//字符串转换函数，将其他变量转换为字符串

```
int sprintf(char *buf, char *format [, argument]...);
```



D01-03: 根据输入整数决定输出小数点位数

◆ D01-03: 由参数确定输出的小数位数

✓从标准输入读入浮点数 x ($-10 < x < 10$)和整数 m ($0 < m < 13$), 在标准输出上输出 $\sin(x)$ 的值, 保留到小数点后 m 位数字, 如:

➤输入: 3.14 3, 输出: 0.002;

➤输入: 3.14 10, 输出: 0.0015926529

◆ 问题分析

✓如果直接用`printf("%.#f", sin(x))`, 则需要用`switch`或`if`语句, 有很多判断条件 (这里`#`是常数, 值跟输入的 m 相同)

✓利用`sprintf`函数, 根据输入的 m 构造格式字符串



使用sprintf生成字符串

```
#include <stdio.h>
int main()
{
    int m; // 小数点位数
    double x; // 输入x
    char format[32]; // 要存储的格式串
    scanf("%lf%d", &x, &m);
    // 利用sprintf生成格式串
    sprintf(format, "%%.%df\n", m);
    // 利用生成的格式串控制输出格式
    printf(format, sin(x));
}
```

若输入m位3，则"%%.%df\n"变为"**%.3f\n**"，且该字符串存入字符数组format，利用该字符串控制输出格式

说明：斜杠是编译器级别的转义，%是printf内部的解析特殊符号，因此斜杠是不行的，只能是%%，不能是\%

sprintf 函数非常好用，任何可以输出到屏幕的数据都可以转换为字符串



使用字符串转换函数：sscanf

- ◆从字符串中转换出各种类型的变量，与sprintf对应
 - ✓scanf从键盘读入一串符号，将其存入到不同变量中
 - ✓sscanf用法与scanf一样，只不过不是从键盘输入，而是直接从一个给定的字符串中读入
 - ✓比scanf多一个参数（第一个参数），就是要存储的要读取的字符串，后面参数的使用与printf完全

//字符串转换函数，从字符串中提取各种变量

```
int sscanf(const char *buf, char *format [, arg]...);
```




D01-04: 从一行字符串中提取变量

◆ D01-04: 提取日期和时间

- ✓ 计算机显示的时间通常有特殊的格式，比如计算机给出的格式 `17/Apr/2018:10:28:28 +0800`，表示北京时间2018年4月17日10时+28分28秒。给出一个这种格式表示的字符串，提取其中的每一项，并在屏幕上逐行显示出来

◆ 问题分析

- ✓ 定义各种类型的输入变量
- ✓ 编写格式控制符，从字符串中读入变量的值



使用sscanf从字符串中提取变量

```
#include <stdio.h>
int main()
{
    int day, year, h, m, s;
    char mon[4], zone[6];
    char buf[] = "17/Apr/2018:10:28:28 +0800"; //要转换的字符串
    //从字符串中, 按照指定的格式读入日期的各个部分
    sscanf(buf, "%d/%3c/%d:%d:%d:%d %s",
           &day, mon, &year, &h, &m, &s, zone);
    mon[3] = '\0'; //理解此句话的作用
    printf("%d\n%s\n%d\n%d\n%d\n%d\n%s",
           year, mon, day, h, m, s, zone); //按照格式输出
    return 0;
}
```



2018
Apr
4
17
28
28
+0800



D01-05综合应用：扩展字符

- ◆【问题描述】从键盘输入包含扩展符'-'的字符串，将其扩展为等价的完整字符，例如将a-d扩展为abcd，并输出扩展后的字符串。
 - ✓要求：只处理[a-z]、[A-Z]、[0-9]范围内的字符扩展，即只有当扩展符前后的字符同时是小写字母、大写字母或数字，并且扩展符后的字符大于扩展符前的字符时才进行扩展，其它情况不进行扩展，原样输出。例如：a-R、D-e、0-b、4-B等字符串都不进行扩展。
- ◆【输入形式】：从键盘输入包含扩展符的字符串
- ◆【输出形式】：输出扩展后的字符串
- ◆【输入样例1】：ADEa-g-m02
- ◆【输出样例1】：ADEabcdefghijklmnopqrstuvwxyz02
- ◆【输入样例2】：cdeT-bcd
- ◆【输出样例2】：cdeT-bcd
- ◆【样例说明】
 - ✓将样例1的输入ADEa-g-m02扩展为：ADEabcdefghijklmnopqrstuvwxyz02；样例2的输入cdeT-bcd中，扩展符前的字符为大写字母，扩展符后的字符为小写字母，不在同一范围内，所以不扩展



问题分析

◆基本字符（串）处理

✓读写字符串：gets、puts

◆判断是否需要需要进行字符串扩展

```
//判断是否需要扩展字符串
```

```
int isexpand(char start, char end);
```

◆字符串扩展

```
//扩展字符串
```

```
for (ch = src[i] + 1; ch < src[i + 2]; ch++)  
    dest[j++] = ch;
```



代码实现：主函数和判断是否扩展函数

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define LEN 1024
//判断是否需要扩展字符串
int isexpand(char start, char end);
//将源串扩展为目标串
char *expand(const char *src, char *dest);
int main()
{
    char src[LEN], dest[LEN];
    gets(src);
    expand(src, dest);
    puts(dest);
    return 0;
}
```

```
//判断是否需要扩展字符串
int isexpand(char start, char end)
{
    if (islower(start) && islower(end) && start < end)
        return 1;
    if (isupper(start) && isupper(end) && start < end)
        return 1;
    if (isdigit(start) && isdigit(end) && start < end)
        return 1;
    return 0;
}
```



字符串查找：库函数

◆子串查找函数： strstr

```
//在字符串中查询是否存在子字符串sub_str  
//返回中第一次出现的位置指针  
//如果不存在，则返回NULL。
```

```
char *strstr(char *s, char *sub_str);
```

◆D01-06：字符串替换

✓从标准输入读入数据中，每行最多包含一个字符串 "_xy_". 将输入行中的"_xy_"替换为"_abc_"，在标准输出上输出替换后的结果

◆问题分析

✓如何找到"_xy_"： strstr?

✓找到后，如何替换为 "_abc_"： 数组操作、指针运算?



D01-06: 字符串替换

提示: BUFSIZ是一个常用的说明缓冲区大小的符号常量, 实际数量取决于编译系统的版本和编译选项, 常见的数值为512或4096

```
#include <stdio.h>
#include <string.h>
int main(){
    char buf[BUFSIZ], *p, *str = "_xy_";
    while (gets(buf) != NULL){//读入字符串
        p = strstr(buf, str);//字符串中查询子串
        if (p == NULL){//找不到子串, 直接输出
            printf("%s", buf);
            continue;
        }
        *p = '\0';//将找到的位置设置为\0, 从而输出前面的串
        //用%s输出时, 通过buf只能输出前面的串
        //代码中直接输出_abc_子串
        //p+strlen(str)通过指针运算, 输出_xy_后面的串
        printf("%s_abc_%s", buf, p + strlen(str));
    }
    return 0;
}
```

输出示例 abc_xy_ef

buf a b c _ x y _ e f \0

被替换的字符不输出
其它字符照常输出

用指针指向开始输出的位置

buf a b c \0 x y _ e f \0
↑ p ↑ p+strlen(str)

从子串 "_xy_" 的第一个字符将输入分为两部分

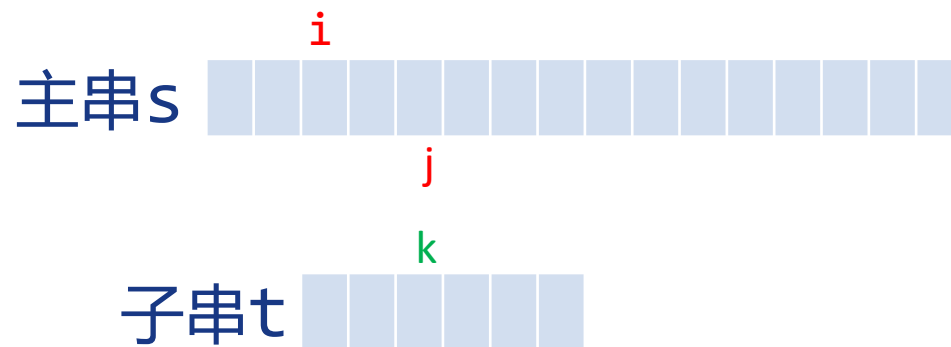


字符串查找：strstr函数的代码实现

◆使用C代码实现（在字符串s中查找t）

```
int index(char s[], char t[])
{
    int i, j, k;
    for (i = 0; s[i] != '\0'; i++)
    {
        for (j = i, k = 0;
             t[k] != '\0' && s[j] == t[k];
             j++, k++) ;

        if (t[k] == '\0')
            return i;
    }
    return -1;
}
```





思考1-不区分大小写

- ◆目前实现了大小写相关的字符串查找，即字符串"the"和"The"是不同字符串
- ◆如何实现大小写无关的字符串查找？

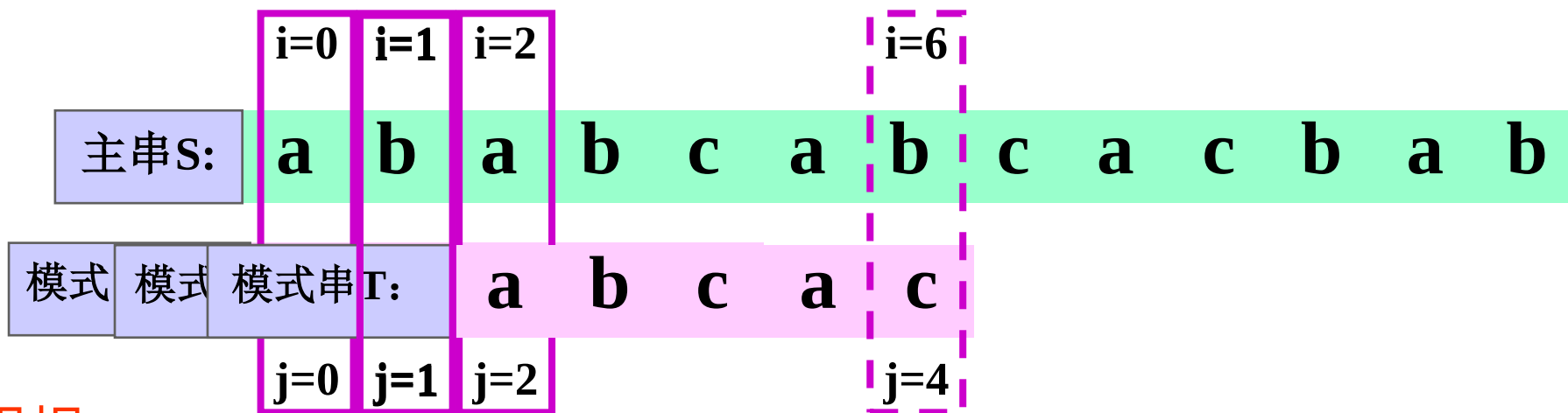
在比较字符时，可将要比较字符均转换为小写或大写即可实现大小写无关查找

设函数char tolower(char c)用于将字符c转换为相应小写字符，则上面index可改为

```
int index(char s[ ], char t[ ]){
    int i, j, k;
    for(i = 0; s[i] != '\0'; i++){
        for(j=i, k=0;
            t[k]!='\0' && tolower(s[j]) == tolower(t[k]);
            j++, k++) ;
        if(t[k] == '\0')
            return ( i);
    }
    return ( -1);
}
```



思考2-字符串的查找算法：串的模式匹配



算法的基本思想:

从主串 s 的第 i 个字符起和模式串的第一个字符比较之:

- 1) 若相等, 则继续逐个比较后续字符; 否则从主串的下一个字符起再重新和模式串的第一个字符比较之
- 2) 依此类推, 直至模式串 t 中的每一个字符依次和主串 s 中的一个连续字符序列相等, 则称匹配成功, 函数值为和模式 T 中第一个字符相等的字符在主串 S 中的位置
- 3) 否则称匹配不成功, 函数返回-1



思考2-简单模式匹配算法实现：暴力破解

```
int index(char s[], char t[])
{
    int i, j, k;
    for (i = 0; s[i] != '\0'; i++)
    {
        for (j = i, k = 0;
             t[k] != '\0' && s[j] == t[k];
             j++, k++) ;

        if (t[k] == '\0')
            return (i);
    }
    return (-1);
}
```

显然，在最坏情况下（被查找的串不存在），该算法语句的最大执行频度为： $N*M$ 。
 N 为源串的长度， M 为要查找的串的长度

时间复杂度为 **$O(n*m)$**



暴力破解 (BF, Brute Force) 法的另一种实现

◆index查找函数使用两层循环，可改为只是用一层循环的字符串查找算法，但时间复杂度并没有变化

```
int index(char s[], char t[]){
    int i=0, j=0;    //设置查找的起始位置
    while(s[i] != '\0' && t[j] != '\0') {
        if(s[i] == t[j]) { //若字符相等，继续查找下一个字符
            i++; j++;
        }
        else { //若字符不等，则s中退回到上次查找开始的下一个位置
            i = i-j+1;
            j = 0;
        }
    }
    if(t[j] == '\0') //查找到字符串t，返回t在s中的起始位置
        return i-j;
    else
        return -1;
}
```



思考3-更高效的模式匹配算法

- ◆暴力破解方法在每次查找失败后回溯到主串上次查找起始位置的下一个位置，之后从头开始匹配的子串，之前匹配的信息并没有在以后使用
- ◆KMP (Knuth-Morris-Pratt) 算法
 - ✓源串称为主串，定义为S，当前匹配位置为i；目标串称为子串，定义为T，当前匹配位置为j
 - ✓当前匹配在找到不匹配的字符后，重新开始匹配时：
 - 主串当前位置i不回溯，即不重置为上次匹配开始位置的下一位置
 - 子串当前位置j视情况回溯至起始串位置或子串中某一位置



KMP算法核心思想：计算子串回溯位置

根据子串T当前匹配的规律： $T_0 \dots T_{k-1} = T_{j-k} \dots T_{j-1}$
由当前失配位置j(已知)，可以归纳计算下次匹配起点k的表达式。

令 $k = \text{next}[j]$ (**函数next用子串当前位置j来计算下次开始匹配位置k**，k与j显然具有函数关系)，则

$$\text{next}[j] = \begin{cases} -1 & \text{当 } j = 0 \text{ 时} \\ \max \{ k \mid 0 < k < j \text{ 且 } 'T_0 \dots T_{k-1}' = 'T_{j-k} \dots T_{j-1}' \} & \\ 0 & \text{其他情况} \end{cases}$$

取当前位置j前序串首尾最大的相同子串长度

注意：

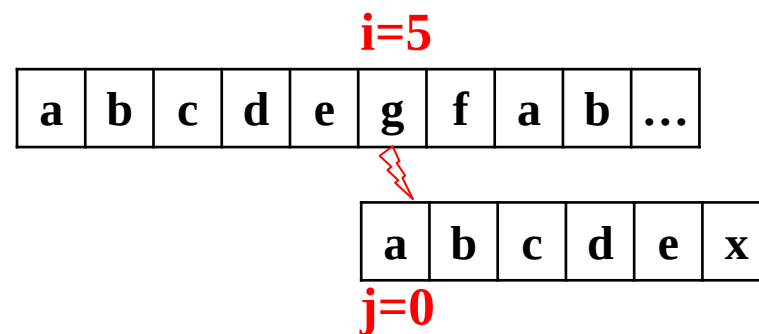
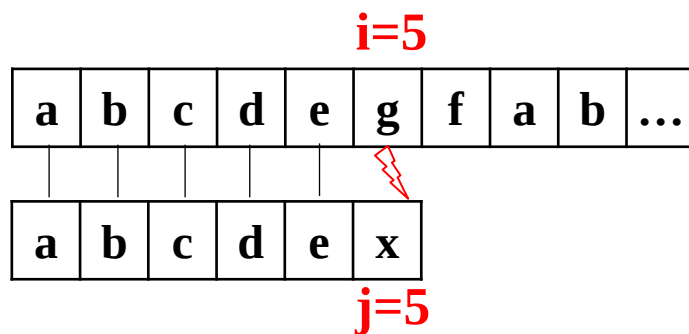
- (1) k值仅取决于子串本身而与相匹配的主串无关。
- (2) k值为子串从头向后及从j向前的两分子串最大相同子串的长度。
- (3) 这里的两分子串可以有部分重叠的字符，但不可以全部重叠，即k最大为j-1。



KMP算法核心思想

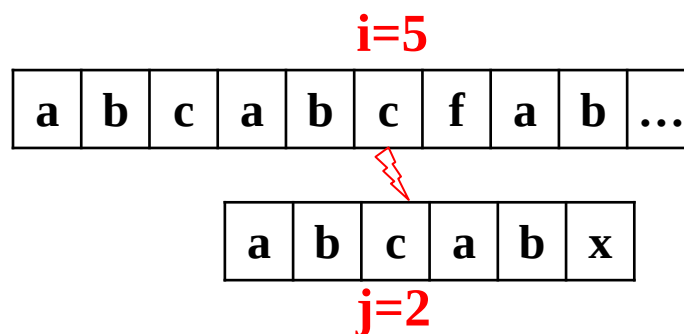
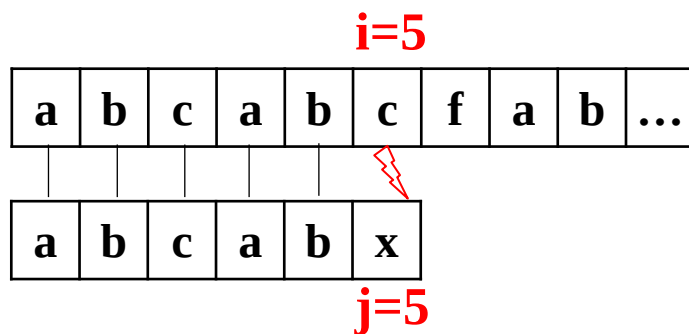
1. 子串T中无重复子串

j	0	1	2	3	4	5
子串T	a	b	c	d	e	x
next[j]	-1	0	0	0	0	0



2. 子串T中有重复子串

j	0	1	2	3	4	5
子串T	a	b	c	a	b	x
next[j]	-1	0	0	0	1	2





KMP算法C语言实现

```
int KMPindex(char S[ ], char T[ ]) {  
    int i = 0, j=0, *next;  
    next = (int *)malloc(sizeof(int)*(strlen(T)+1));  
    getnext(T, next); //求next  
    while ( S[i]!='\0' && T[j]!='\0') {  
        if (S[i] == T[j]) {  
            i++;  
            j++;  
        }  
        else  
            //j回退到相应位置开始匹配, i不变  
            (j == 0) ? i++ : ( j = next[j]);  
    }  
    free(next);  
    if ( T[j] == '\0') //匹配成功, 返回匹配位置  
        return i-j;  
    else  
        return -1;  
}
```

算法分析:

由于指针i无须回溯, 比较次数仅为n, 加上计算next[j]时所用的比较次数m, 比较总次数也仅为n+m, 时间复杂度为 $O(n+m)$, 大大优于朴素的Brute-Force算法



KMP算法C代码实现：计算next

```
void getNext(char T[], int next[])
{
    int i = 0, j = -1;
    next[0] = -1;
    while (T[i] != '\0')
    {
        if (j == -1 || T[i] == T[j])
        { // i为后缀位置; j为前缀位置
            i++;
            j++;
            next[i] = j;
        }
        else
            j = next[j]; //若字符不同, 则j值回溯
    }
}
```



思考4-更复杂的字符串查找

- ◆注意：index函数只能查找子字符串首次出现，如果一行中含有多个要查找的字符串怎么办？(类似Office软件中的查找功能)
 - ✓如何查找子字符串的最后一次出现？
 - ✓如果要查找一个字符串在一个文件中的所有出现（如给出所有出现的行列位置），如何实现？
- ◆在一个文件中查找给定串并用另一个替换串，如何实现？(Office软件中的替换功能)
- ◆如何实现模糊查找（如UNIX命令grep），如要查找的串形式为：
comp?ter, com*er (?单字符匹配, *多字符匹配)



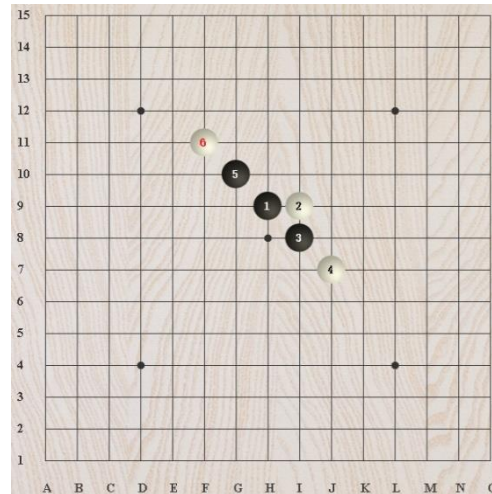
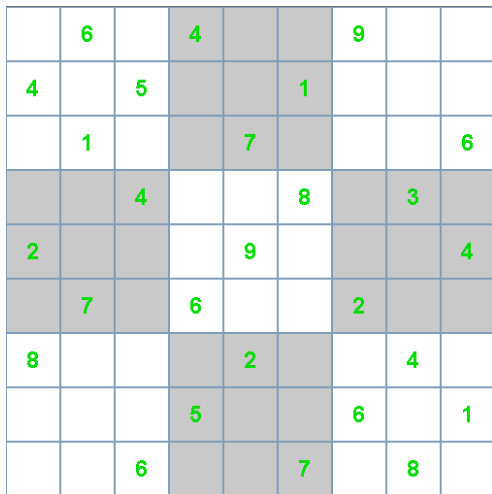
提纲：数据结构程序设计基础（C语言）

- ◆ 1.1 字符串
- ◆ 1.2 二维（多维）数组
- ◆ 1.3 指针
- ◆ 1.4 结构和联合
- ◆ 1.5 文件处理



二维（多维）数组

- ◆一维数组（单下标数组）：字符串，语音信号， $\sin(t_k)$, ...
- ◆二维数组（双下标）应用更广泛：平面图像，excel表格，



	A	B	C	D	E	F	G	H
1	排名	用户	学号	得分	罚时	A	B	C
2						922/1235	916/1033	828/927
3	1	刘冰	#####	800	13:57:51	0:51:09	1:05:29	0:57:33
4	2	乌云	#####	800	15:31:30	0:27:08(+1)	1:58:11	1:49:46
5	3	校	#####	800	16:59:24	1:26:11(+3)	1:32:18	1:43:33
6	4	俊	#####	800	21:33:04	1:16:00	1:52:45	2:43:37(+1)
7	5	柯	#####	800	23:30:58	3:18:20(+4)	4:28:36(+1)	3:34:32
8	6	玮	#####	800	24:03:40	2:42:19	3:14:24	2:06:48(+1)
9	7	超	#####	800	24:32:52	3:24:27(+3)	1:29:11	2:56:31(+1)
10	8	昕	#####	800	25:43:39	0:52:54(+3)	0:56:25	1:48:14
11	9	一	#####	800	25:53:40	3:43:44(+1)	2:22:09	2:44:14
12	10	怀	#####	800	27:57:44	3:47:20(+2)	2:20:19	2:51:30



二维数组

◆二维下标数组常用于表示由行和列组成的表格，为了表示表格中的元素，需要指定两个下标

✓第一个维度表示元素的行，第二个维度表示元素的列

◆定义双下标数组示例： `int a[5][4];`

✓数组 a 包含 5 行 4 列，称为 5×4数组

◆一般地，m行n列数组称为m×n数组，语句中引用某个元素应写为`a[i][j]`，而不是`a[i, j]`

◆`a[i][j]` 表示数组中的第 $i*n+j$ 个元素

<code>a[0][0]</code>	<code>a[0][1]</code>		
		<code>a[2][2]</code>	
		<code>a[3][2]</code>	
			<code>a[4][3]</code>

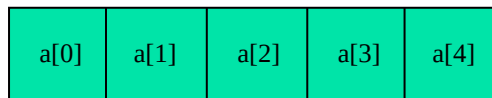
```
int a[5][4];
for(i=0; i<5; i++)
{
    for(j=0; j<4; j++)
    {
        a[i][j] = i*4 + j;
        printf(" %d ", a[i][j]);
    }
    printf("\n");
}
```



多维数组

- ◆ 二维数组可以看作是一个元素为另一个一维数组的一维数组；
三维数组可以看作元素为二维数组的一维数组， ...
- ◆ 多下标数组可以有多个下标

✓ $a[5]$

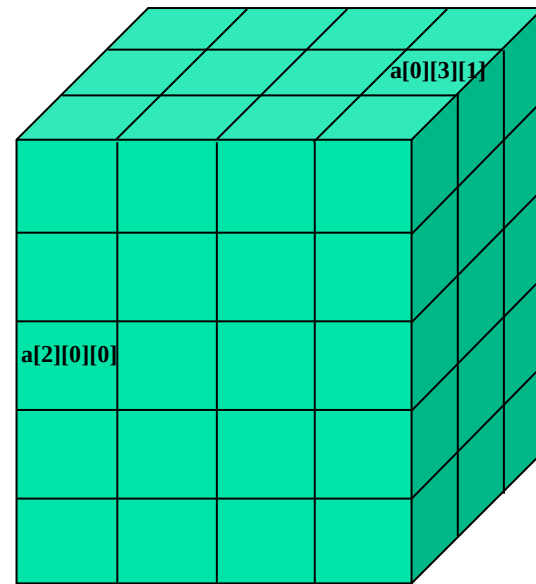
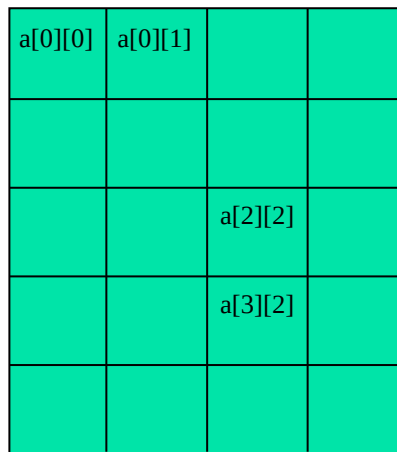


✓ $a[5][4]$

✓ $a[5][4][3]$

✓ $a[X][Y][Z][A]$

✓ $a[M][N][K][L][P]$





二维数组的排列和初始化

- ◆ C语言的数组名都表示数组的首地址，数组元素从首地址开始，按顺序连续在地址中存储
- ◆ 二维数组的存储是行优先的，下面的两种初始化方式是一样

```
int y[4][3] = {  
    {1, 3, 5},  
    {2, 4, 6},  
    {3, 5, 7},  
    {0, 0, 0}};  
int y[4][3] = { 1, 3, 5, 2, 4, 6, 3, 5, 7, 0, 0, 0 };
```

- ◆ 由于C语言不对数组的下标做严格的范围检查，访问数组元素 `y[1][1]` 也可以写为 `y[0][4]`，建议访问数组元素时，行、列的编号都限制在定义时的行、列的范围内



二维（多维）数组作为参数

- ◆如果把一个二维数组作为参数，则在函数定义中，形参数组的说明中必须指明列的数目，而行的数目可空着不写
- ◆以此类推，对于三维及以上数组，其作为参数传递时，形参说明中只能省略最内层的维数

```
int main()  
{  
    float a[4][3], b[3][4], c[4][4];  
    ... fun(a, b, c);  
    ...  
}  
void fun(float x[][3], float y[][4], float z[][4])  
{  
    ...  
}
```




D01-07综合应用：计算小岛面积

【问题描述】 用一个二维方阵（最小为3X3，最大为50X50）表示一片海域。方阵中的元素只由0和1组成。1表示海岸线。计算由海岸线围起来的小岛面积（即：由1围起来的区域中0的个数）。样例所示8X8方阵表示的小岛面积为9，上述方阵表示的海域满足下面两个要求：

- 1、小岛只有一个。
- 2、用1表示的海岸线是封闭的，但有可能是凸的，也有可能是凹的。

【输入形式】 先从标准输入中输入方阵的阶数，然后从下一行开始输入方阵的元素（只会0或1）。

【输出形式】 在标准输出上输出用整数表示的小岛面积。

【输入样例】

```
8
0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 1 0 0
0 0 1 0 0 0 1 0
0 1 0 0 0 1 0 0
0 1 0 1 0 1 0 0
0 1 1 0 1 0 0 0
0 0 0 0 0 0 0 0
```

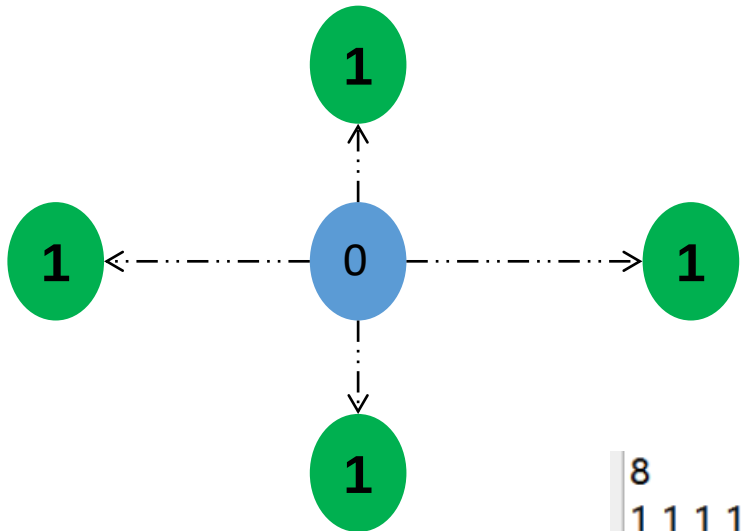
【输出样例】

9



D01-07: 算法分析

对于方阵中的任意一个元素0，如果其位于同一行上的两个1之间，并且位于同一列上的两个1之间，则该元素在1围起来的区域中 (?)



说明：目前的算法其实存在问题，有些特殊的情况可能会计算错误，需要设计更完善的算法！如右图

```
8
1 1 1 1 1 1 1 1
1 0 0 0 0 0 0 1
1 0 1 1 1 1 0 1
1 0 1 0 0 1 1 1
1 0 1 0 0 0 0 0
1 0 1 1 1 1 1 1
1 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1
```

```
#include <stdio.h>
```

```
int main(){
```

```
int a[50][50] = {0}, i, j, k, c, n, area = 0;
```

```
scanf("%d", &n);
```

```
for (i = 0; i < n; i++)
```

```
for (j = 0; j < n; j++)
```

```
scanf("%d", &a[i][j]);
```

```
for (i = 0; i < n; i++)
```

```
for (j = 0; j < n; j++) {
```

```
c = 0;
```

```
if (a[i][j] == 0) {
```

```
for (k = i; k < n; k++)
```

```
if (a[k][j] == 1){c++;break;}
```

```
for (k = i; k >= 0; k--)
```

```
if (a[k][j] == 1){c++;break;}
```

```
for (k = j; k < n; k++)
```

```
if (a[i][k] == 1){c++;break;}
```

```
for (k = j; k >= 0; k--)
```

```
if (a[i][k] == 1){c++;break;}
```

```
}
```

```
if (c == 4) area++;
```

```
}
```

```
printf("%d", area);
```

```
return 0;
```

```
}
```



提纲：数据结构程序设计基础（C语言）

◆ 1.1 字符串

◆ 1.2 二维（多维）数组

◆ 1.3 指针

✓ 1.3.1 复习：指针基础

✓ 1.3.2 指针数组

✓ 1.3.3 函数指针

◆ 1.4 结构和联合

◆ 1.5 文件处理



1.3.1 指针

- ◆ 指针表示另外一个数据项的地址
 - ✓ 指针变量是存放所指对象地址的变量
- ◆ 允许指针指向任何类型的对象（可指向基本类型、构造类型），甚至可指向其它指针或指向函数
- ◆ 当对象本身不能被直接传送的情况下，可以通过指针来进行传递
- ◆ 指针应具有非零（无符号整数）值
 - ✓ 如将0（通常#define NULL 0）赋给予指针，则该指针没有指向任何具体对象，即空指针
- ◆ 指针使用得较多，指针用好了，可使程序表达能力大大加强
 - ✓ 使用时需多加小心，要切实掌握指针含义和用法，否则会使程序运行时乱套
 - ✓ OJ、Judge上出现SIG开头的错误，基本上都是指针的问题



指针的定义

◆指针变量的定义（说明）

<类型> * <变量>;

◆指针是用所指对象类型来表征的。如：

```
int    *px;    /* 指向整型的指针 */
char   *pc;    /* 指向字符型的指针 */
char   *acp[10]; /* 由指向字符的指针构成的数组，即指针数组 */
char   (*pac)[10]; /* 指向字符数组的指针，即数组指针 */
int    *fpi( ); /* 返回值为指向整型的指针的函数，指针函数 */
int    (*pfi)(); /* 指向一个返回值为整型的函数的指针，函数指针 */
```



指针运算符

◆单目（取地址）运算符 &

✓用来取变量或数组成员地址，即获得某个变量的指针

◆单目（间接引用，或递引用）运算符 *

✓用来取某地址中内容，即获取指针所指对象

◆例

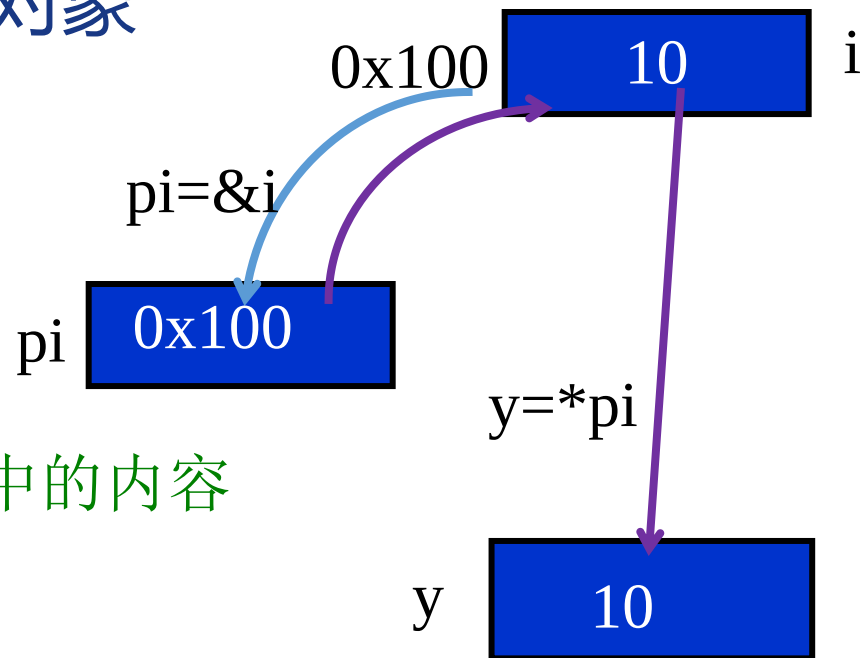
```
int i = 10, y=20, *pi;
```

```
//将变量i的地址赋给指针变量pi，称pi指向i
```

```
pi = &i;
```

```
//取pi所指对象的值赋给y，即取pi中所存地址中的内容
```

```
y = *pi;
```





务必保证指针指向合法的空间

- ◆注意：使用任何指针变量之前必须先给它赋一个所指合法具体对象的地址值。
- ◆下面两种用法都是错误的

1) `int x = 1;`

`int *px;`

`*px = x;`

这是C程序中指针最常犯的错误！

2) `char *string;`

`scanf("%s", string);`

`strcpy(string, "Hello");`



务必保证指针指向合法的空间

◆通过&运算符使指针指向某个对象

```
int n=10, *px;  
px = &n;
```

◆将另一个同类型的（已指向具体对象的）指针赋给它以获得值，两指针指向同一对象

```
int n=10, *px,*py;  
char str[10],*pstr;  
px = &n; py = px;  
pstr = str;
```

◆使用malloc或alloc等函数给指针分配一个具体空间（动态存储分配）

```
p = (char *)malloc(strlen(s)+1);
```




动态内存管理 (malloc与free)

- ◆在C中可以使用标准库函数malloc动态为指针变量申请一块内存空间（以字节为单位）（用于初始化指针变量），并返回该空间首地址

函数原型为： `void * malloc ( size_t size );`

- ◆使用malloc初始化指针变量的常见用法：

```
char *s;
```

```
int *intptr;
```

```
s = (char *) malloc(32);    //s指向大小为32个字节（字符）的空间
```

```
s = (char *) malloc(strlen(p)+1); //s指向能正好存放字符串p的空间
```

```
intptr = (int *)malloc(sizeof(int)*10); //ptr指向存放10个整型元素空间
```

- ◆使用malloc申请到的动态空间在不用时应使用函数free释放。如，
`free(s);`
- ◆头文件： `#include <stdlib.h>`



指针运算

◆指针和整型量可以进行**加减**

- ✓若p为指针，则 $p+n$ 和 $p-n$ 是合法的，同样 $p++$ 也是合法的，它们的结果同指针所指对象类型相关
- ✓如果p是指向数组某一元素的指针，则 $p+1$ 及 $p++$ 为数组下一元素的指针

◆当P1和P2指向同一类型时，可以进行**赋值**

- ✓如： $py = px$ ，则px，py指向同一对象。（注意：与strcpy(py, px)的不同。）

◆两个指向同一类型的指针，可进行 $=$ ， $>$ ， $<$ 等**关系运算**，其实就是地址的比较

- ✓注意：两个同一类型的指针不能相加

◆两个指向同一数组成员的指针可进行**相减**，其结果为两指针间相差元素的个数

- ✓如：p指向数组第一个元素，q指向数组最后一个元素，则 $q - p + 1$ 表示数组长度



指针运算 (续)

◆思考?

- ✓ $p++$ 和 $p+1$ 的区别
- ✓ $y = *px + 1$ 和 $y = *(px + 1)$ 的区别
- ✓ $y = (*px)++$ 和 $y = *px++$ 的区别

```
int a[5] = {0, 1, 2, 3, 4};  
int *pi;  
pi = &a[0]; //pi指向a[0]  
pi++;      //pi指向a[1], *pi为a[1], 即1  
pi+=2;     //pi指向a[3], *pi为a[3], 即3
```

```
int a[10];  
int *pi= &a[0];  
char str[10];  
char *pc= &str[0];  
pi++;      //pi增加一个单位, 即int类型的4个字节  
pc++;      //pc增加一个单位, 即char类型的1个字节
```



D01-08: 输出输入行中的最长行

◆ 数据结构设计

✓ 设两个一维字符数组来存储新输入行及当前最长行

◆ 主算法设计

while (还有新输入行)

if (新行比以前保存的最长行更长)

保存新行及其长度;

输出所保存的最长行;

如何从标准输入中输入一行？
如何判断输入结束？

`while(gets(s)!=NULL)`

...

如何比较两个字符串长度大小？
需要计算字符串长度：

`int str_len(char s[ ]);`

如何保存一个字符串？
需要拷贝一个串至另一个串：

`int str_copy(char s[], char t[]);`



D01-06: 代码实现 (数组方式)

```
#include <stdio.h>
#include <string.h>
#define MAXLINE 1024
int main(){
    int len, max; //当前和最长字符串长度
    char line[MAXLINE]; //当前输入字符串
    char save[MAXLINE]; //存储的最长字符串
    max=0;
    while (gets(line) !=NULL) {
        len=strlen(line);
        if (len>max) {
            max=len;
            str_copy(save,line);
        }
    }
    if (max>0) printf("%s",save);
    return 0;
}
```



使用标准字符串处理函数

◆ #include <string.h>

- ✓ `int strlen(char s[]);` /*计算字符串长度, 字符串以\0结束*/
- ✓ `char *strcpy(char s[], char t[]);` /*将字符串t拷贝到字符串s中*/
- ✓ `char *strcat(char s[], char t[]);` /*将字符串t拷贝到字符串s尾部*/
- ✓ `int strcmp(char s[], char t[]);` /*比较两个字符串, 若 $s > t$, 则返回大于0的数; 若 $s < t$, 则返回小于0的数; 若相等, 返回0 */

◆ C还提供的高效函数接口 (不仅仅用于字符串)

- ✓ `void *memcpy(void *s, void *t, int n);`
- ✓ `void *memmove(void *s, void *t, int n);`
- ✓ `int memcmp(void *s, void *t, int n);`



D01-08: 指针方式实现

◆数组实现时需要频繁拷贝字符串，效率较低

◆算法设计

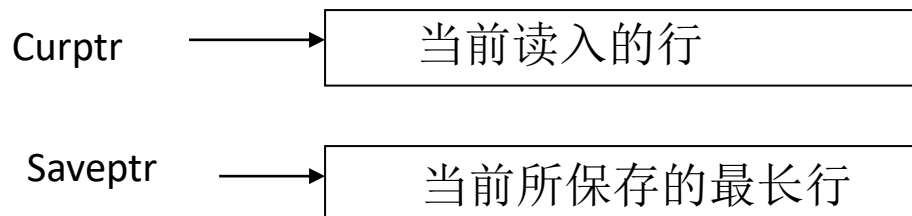
设指针变量Curptr和Saveptr分别指向当前行（新行）和当前最长行

while(还有新输入行)

if (Curptr所指向的行比Saveptr所指向的行长)

交换Curptr和Saveptr指针并保存新行长度；

输出Saveptr所指内容





D01-08: 代码实现 (指针方式: 无字符串拷贝)

初始化指针使其分别指向一个数组

保存新行长度

交换指向当前行和所保存行的指针

```
#include <stdio.h>
#include <string.h>
#define MAXLINE 1024
int main() {
    int len, max;           //当前和最大字符串的长度
    //指针: 指向输入、存储和临时交换字符串
    char *curptr, *saveptr, *tmp;
    char save1[MAXLINE], save2[MAXLINE];
    curptr = &save1[0];
    saveptr = &save2[0];
    max = 0;
    while (gets(curptr) != NULL) {
        len = strlen(curptr);
        if (len > max) {
            max = len; tmp = curptr;
            curptr = saveptr; saveptr = tmp;
        }
    }
    if (max > 0) printf("%s", saveptr);
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
#define MAXLINE 1024
int main(){
    int len, max; //当前和最长字符串长度
    char line[MAXLINE]; //当前输入字符串
    char save[MAXLINE]; //存储最长字符串
    max=0;
    while (gets(line) !=NULL) {
        len=strlen(line);
        if (len>max) {
            max=len;
            str_copy(save,line);
        }
    }
    if (max>0) printf("%s",save);
    return 0;
}
```

数组实现



指针作为函数参数

◆在C中函数参数传递方式为"传值"

- ✓最大好处是函数调用不会改变实参变量的值
- ✓如何通过函数调用来改变实参变量的值?
 - 如：通过函数swap(a,b)来交换两个变量的内容

◆可以通过将指针作为函数参数来改变实参内容

```
void swap(int *px, int *py){  
    int temp;  
    temp = *px;  
    *px = *py;  
    *py = temp;  
}  
  
int main() {  
    int a = 2, b = 3;  
    swap(&a, &b);  
}
```



1.3.2 指针和数组

- ◆ 数组就是一个（常量）指针，数组的名就是指向数据第一个元素的指针（首地址）
- ◆ 数组可以按指针方式使用，如：
 - ✓ `int a[10], *p;`
 - ✓ `for(i=0; i<n; i++) ...*(a+i)...`
- ◆ 指针亦可按数组形式访问，如：
 - ✓ `void fun(int *p){ ...p[i]...`
- ◆ 数组名可作为参数进行传递，当将数组名传给函数时，实际上所传递的是数组的首地址（即数组第一个元素的地址）



思考：指针和数组

```
#include <stdio.h>
int main()
{
    int a[10];
    int *p;
    int (*pp)[10];
    p = a;
    //p = &a; //cannot convert 'int (*)[10]' to 'int*' in initialization
    //pp = a; //cannot convert 'int [10]' to 'int (*)[10]' in assignment
    pp = &a;
    printf("a=%x, &a=%x, &a[0]=%x, p=%x, pp=%x, sizeofa=%d, sizeofp=%d\n",
           a, &a, &a[0], p, pp, sizeof(a), sizeof(p));
    return 0;
}
```

a=62fde0, &a=62fde0, &a[0]=62fde0, p=62fde0, pp=62fde0, sizeofa=40, sizeofp=8



使用指针和数组两种方式实现strcpy函数

◆ (1) 数组方式

```
void strcpy(char s[], char t[])
{
    int i = 0;
    while ((s[i] = t[i]) != '\0')
        i++;
}
```

◆ (2) 指针方式

```
void strcpy(char *s, char *t)
{
    while ((*s = *t) != '\0')
    {
        s++;
        t++;
    }
}
```

```
void strcpy(char *s, char *t)
{
    while (*s++ = *t++)
        ;
}
```



指针数组

- ◆ 单个字符串需要采用一个字符数组来保存，如
`char word[32] = "Wednesday"`
- ◆ 若要同时保存多个相关字符串（通讯录中的人名单、英文字典中单词表、文件中的多行...）？
 - ✓ 如何同时保存如下相关数据：
➤ Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
 - ✓ 方法：
 - 二维字符数组，如：`char days[7][10];`
 - 指针数组，如：`char *days[7];`



指针数组 (续)

◆ 指针数组就是由指针组成的数组，即该数组的每一个元素都是指向某一类型对象的指针

```
char *lineptr[100]; // 由字符指针构成的数组
```

```
int *iptr[50];      // 由整型指针构成的数组
```



指针数组与二维数组的区别

◆二维数组:

```
char days[7][10] = {"Sunday", "Monday", "Tuesday",  
                    "Wednesday", "Thursday", "Friday", "Saturday"};
```

◆存储形式

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
days[0]	S	u	n	d	a	y	\0			
days[1]	M	o	n	d	a	y	\0			
days[2]	T	u	e	s	d	a	y	\0		
days[3]	W	e	d	n	e	s	d	a	y	\0
days[4]	T	h	u	r	s	d	a	y	\0	
days[5]	F	r	i	d	a	y	\0			
days[6]	S	a	t	u	r	d	a	y	\0	

//使用方式:

```
scanf("%s", days[i]);
```

```
gets(days[i]);
```

```
if(strcmp(days[i], "Friday")==0)
```

```
...
```

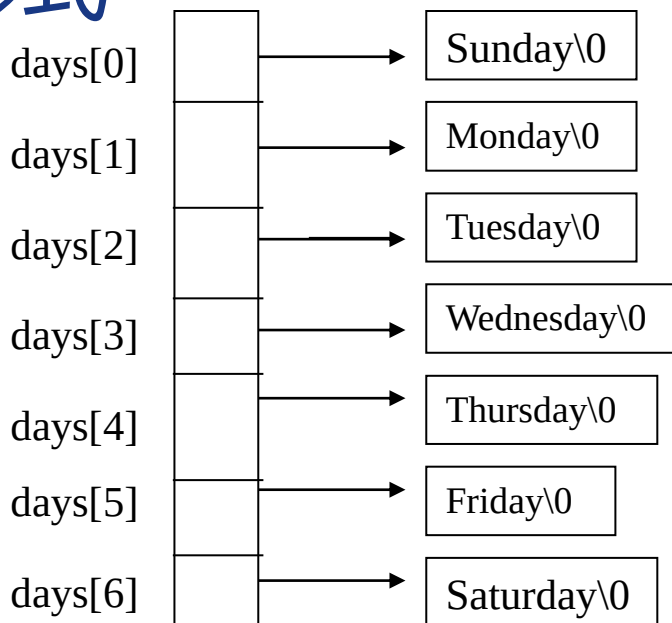


指针数组与二维数组的区别 (续)

◆ 指针数组

```
char days[7][10] = {"Sunday", "Monday", "Tuesday",  
                    "Wednesday", "Thursday", "Friday", "Saturday"};
```

◆ 存储形式



char *days[7]

//使用方式:

```
char buf[32], *days[7];  
scanf("%s", buf);  
days[i] = (char *)malloc(strlen(buf) + 1);  
strcpy(days[i], buf);
```

```
if (strcmp(days[i], buf) == 0)
```

...

//记得要释放分配的空间!!



指针数组与二维数组的区别 (续)

- ◆ 尽管二维字符数组与字符指针数组在存储形式上不同，但它们在初始化形式以及使用方式上却是相同的
 - ✓ 如：无论是指针数组，还是二维数组，下面两种形式访问的都是同一个元素，结果都是字符串"Friday"中的字符'y'
$$*(days[5]+5) = days[5][5] = 'y'$$
 - ✓ 无论是二维数组还是指针数组，`days[5]`访问的都是字符串(即指向字符串的指针)："Friday"
- ◆ 使用指针数组来存放不同长度的字符串可以节省存储空间
 - ✓ 例如：如果要保存从标准输入或文件中读入的行，字符指针数组是一个好的选择。因为读入的行可能长短差异很大。下面程序片段即为保存从标准输入中读入的多行



示例：指针数组的应用

```
// 从输入设备中读入一行后存储
```

```
#define MAXLENGTH 512
```

```
#define MAXLINES 1024
```

```
...
```

```
char *lineptr[MAXLINES], buf[MAXLENGTH];
```

```
int i;
```

```
...
```

```
i = 0;
```

```
while (gets(buf) != NULL)
```

```
{
```

```
    lineptr[i] = (char *)malloc(strlen(buf) + 1);
```

```
    strcpy(lineptr[i], buf);
```

```
    i++;
```

```
}
```

```
...
```

```
//将数字表示的月份转换成英文表示的月份
```

```
char *month_name(int n)
```

```
{
```

```
    static char *name[] = {
```

```
        "illegal month", "January", "February",
```

```
        "March", "April", "May", "June", "July",
```

```
        "August", "September", "October", "November", "December"};
```

```
    return ((n < 1 || n > 12) ? name[0] : name[n]);
```

```
}
```



示例：指针数组的应用：命令行参数

◆C语言中，主函数main还可以带有参数，形式如下：

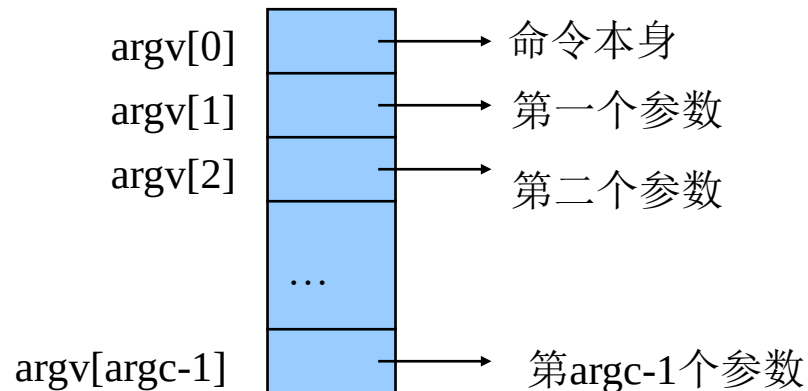
```
int main(int argc, char *argv[])  
// 或  
int main(int argc, char **argv)
```

◆其中：

✓argc包含命令本身在内的参数个数

✓argv指针数组，数组元素为指向各参数（包含命令本身）的指针

```
命令提示符  
d:\YHH\Temp>type echo.c  
#include<stdio.h>  
main(int argc, char *argv[])  
{  
    int i;  
    for(i=1;i<argc; i++)  
        printf("%s%c",argv[i],(i<argc-1)?' ':'\n');  
}  
d:\YHH\Temp>
```



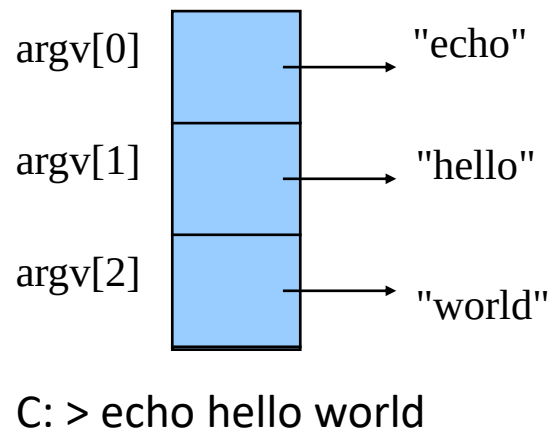
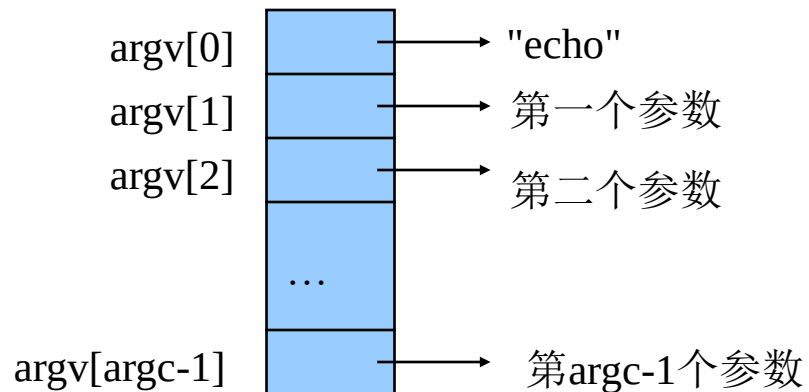


D01-09: echo

◆问题：实现一个命令echo，其将命令后的正文串显示在屏幕上，如：

✓C: > echo hello world

✓屏幕输出：hello world





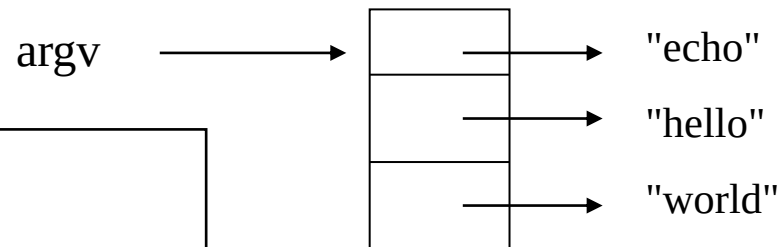
D01-09: 代码实现

◆实现一

```
int main(int argc, char *argv[])
{
    int i;
    for (i = 1; i < argc; i++)
        printf("%s%c", argv[i], (i < argc - 1) ? ' ' : '\n');
    return 0;
}
```

◆实现二

```
int main(int argc, char *argv[])
{
    while (--argc > 0)
        printf((argc > 1) ? "%s " : "%s\n", *++argv);
    return 0;
}
```



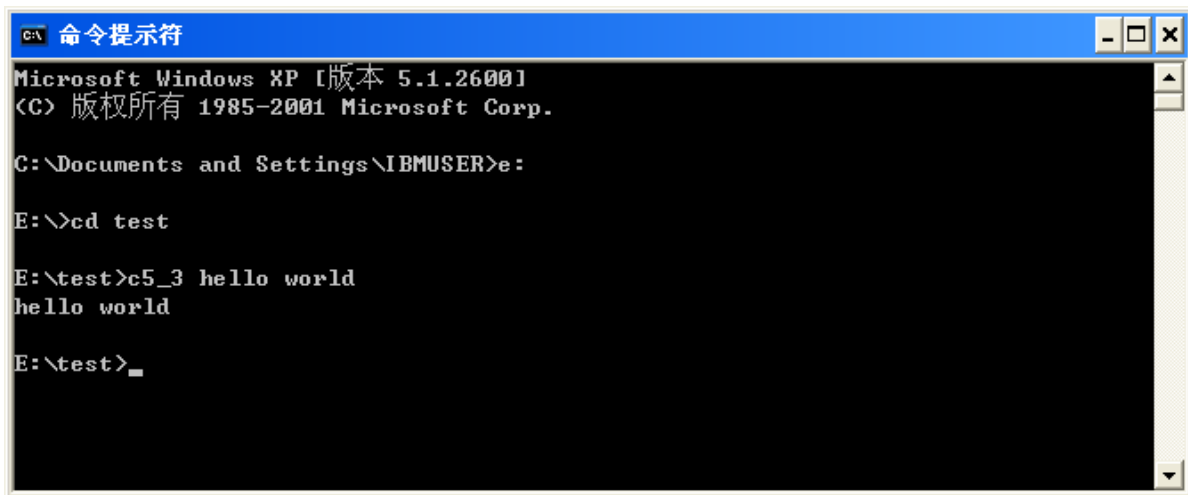


D01-09: 如何运行命令程序

◆方式一：在命令窗口（DOS窗口）中直接运行；

✓若程序名为：echo.exe，文件在\test目录下，则从<开始>菜单中<附件>中找到<命令提示符>，并执行。然后，转到test目录下执行echo.exe文件

◆方式二：在IDE中设置命令行参数，如Dev C++:



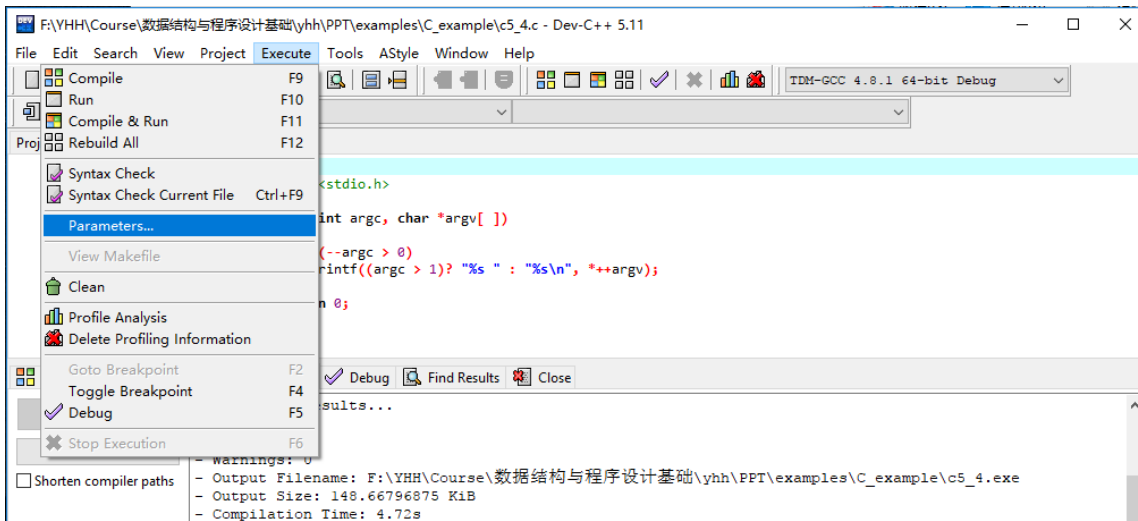
```
命令提示符
Microsoft Windows XP [版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\IBMUSER>e:

E:\>cd test

E:\test>c5_3 hello world
hello world

E:\test>_
```





1.3.3 函数指针

◆函数指针：是一个指针变量，该指针变量可以指向某个函数

```
//f_name是一个函数指针变量，可以指向一个函数  
//这个函数接受一个int型参数，返回一个int型值  
int (*f_name)(int);
```

◆注意与指针函数的区别

✓指针函数是一个函数，函数返回类型的值

```
//strstr是个函数，返回类型为char*  
char *strstr(char *s, char *s1);
```

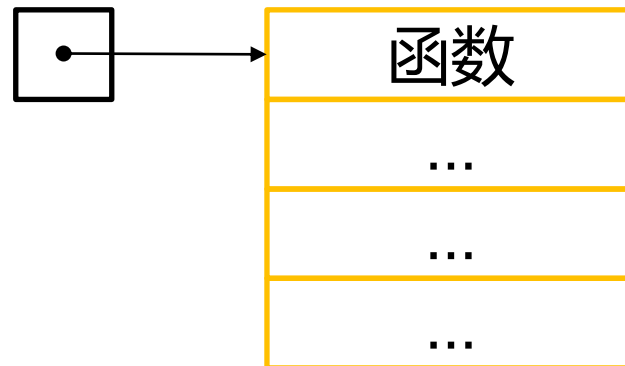
◆类似于指针数组和数组指针的区别

```
char *a[10];    //（一维）指针数组  
char (*a)[10]; //数组指针，可以指向char[10]的数组
```



函数指针

- ◆ 函数指针的类型由函数原型决定，即：函数的参数个数、参数类型和返回类型
- ◆ 函数指针可以指向同类型的函数
 - ✓ 函数名表示的是一个函数的可执行代码的入口地址，也就是指向该函数可执行代码的指针
 - ✓ 函数指针为提高程序描述的能力提供了有力的手段，是实际编程中一种不可或缺的工具





定义函数指针

◆定义一个函数指针变量

<返回类型> (*<标识符>) (<参数表>);

//例如:

```
int (* funPtr1) (int, int);  
void (* funPtr2) (int, int, int);  
int (* funPtr3) (double, char *);  
int * (* funPtr4) ();
```

//声明一个函数指针func,

//可以指向含有两个double参数, 返回double类型的函数

```
double (*func) (double x, double y);
```

// 等价于, 参数的变量名一般省略

```
double (*func) (double, double);
```



使用函数指针

```
double (*func) (double x, double y); //定义一个函数指针
// vs
double sum(double x, double y);      //函数声明
```

```
double sum(double x, double y) { //sum函数定义
    return x + y;
}
```

...

```
func = sum; / 把函数sum赋值给func, func指向sum, 操作func即操作sum
s1 = (*func)(u, v); // 调用, 与sum(u, v)所调用的是同一个函数
s2 = func(u, v);    // 等价于(*func)(u, v)
```

◆为了方便使用, C语言中允许将函数指针变量直接按函数调用的方式使用: `func(u, v)` 与 `(*func)(u, v)` 完全等价!



使用函数指针

- ◆一般函数的参数采用普通数值或指针，函数内部执行与参数类型相关的固定计算方法，对参数进行计算

```
int add(int a, int b); // 普通数值  
int toupper(char* src, char* dst); // 指向变量或数组的指针
```

- ◆利用函数指针，可以设计“动态”绑定的计算函数，实现动态计算方法
- ◆将“动态”绑定的函数以参数形式传递给计算框架函数
 - ✓ “静态”绑定：函数声明（编译）时就已经确定了
 - ✓ “动态”绑定：函数运行时才能确定，且运行时可以变



库函数中的函数指针：快速排序

◆快速排序库函数qsort (stdlib.h头文件)

```
//<stdlib.h>中提供的快速排序函数
//qsort: 负责框架调用和给(*comp)传递所需参数
//根据(*comp)的返回值决定如何交换数组的成员;
//base: 需要排序的数组首地址 (void*指向任意类型的数组);
//num: 参与排序的数组元素的个数;
//wid: 数组中每个元素所占用的字节数;
//comp: 指向数组元素比较函数的指针, 接收两个元素的地址
//(*comp): 函数负责比较两个元素, 返回负数、正数和0,
//分别表示第一个参数先于、后于和等于第二个参数
void qsort( void *base, size_t num, size_t wid,
            int (*comp)(const void *e1, const void *e2) );
```



示例：使用qsort排序

◆给定一个具有n个元素的double型数组，使用qsort()对数组元素按升序和降序排序

```
double a[100]; //需要排序的数组
int n; //数组中实际元素的个数
...
//按升序排序
qsort(a, n, sizeof(double), rise_double);
// 按降序排序
qsort(a, n, sizeof(double), fall_double);
```

```
//升序排序，最基础的写法
int rise_double(const void *p1, const void *p2)
{
    double *d1=(double *)p1; //转换为目标类型
    double *d2=(double *)p2; //转换为目标类型
    if(*d1 < *d2) return -1; //根据大小返回值
    else if(*d1 > *d2) return 1;
    else return 0;
}
```

```
//降序排序，直接类型转换
int fall_double(const void *p1, const void *p2)
{
    if ( *(double *)p1 > *(double *)p2 ) return -1;
    if ( *(double *)p1 < *(double *)p2 ) return 1;
    if ( *(double *)p1 == *(double *)p2 ) return 0;
}
```



示例：使用qsort排序

◆ 下列写法可能存在一些错误

```
//不一定正确的写法，参数直接写目标类型（部分编译器不支持）  
//建议参数采用const void *，在代码中转换为目标类型，更通用！  
int fall_double(const double *p1, const double *p2)  
{  
    if ( *p1 > *p2 ) return -1;  
    if ( *p1 < *p2 ) return 1;  
    if ( *p1 == *p2 ) return 0;  
}
```

```
//肯定错误的写法，思考为什么？  
int rise_double(const void *p1, const void *p2)  
{  
    return (int)(*(double *)p1 - *(double *)p2);  
}  
int fall_double(const double *p1, const double *p2)  
{  
    return (int)(*p1 - *p2);  
}
```

思考：如果数组元素是int类型，可否使用 return *(int*)p1- *(int*)p2?



D01-10: 单词排序

- ◆从键盘读入一组无序的单词，假设每个单词不超过20个字符，总计不超过100个单词，按照字典顺序输出该单词

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int cmp(const void *p1, const void *p2){
    return strcmp((const char*)p1, (const char*)p2);
}
int main(int argc, char *argv[]){
    char name[100][20];
    int i=0, j;
    while(~scanf("%s", name[i])) i=i+1;
    qsort(name, i, sizeof(char)*20, cmp);
    for(j=0; j<i; j++) printf("%s\n", name[j]);
    return 0;
}
```



D01-11: 点集排序

◆C08-07: 点集排序, 已知一个 $N \times 2$ 的二维 double 数组存储二维平面上 N 个点的坐标, 每一行表示一个点, 第一列的数表示 x 的坐标, 第2列表示 y 坐标。使用 `qsort` 函数对这 N 个点先根据 x 坐标值从小到大排序, 如果 x 坐标相同, 则按照 y 坐标值从大到小排序

◆问题分析

- ✓ 用二维数组 `double pt[N][2]` 保存这 N 个点的坐标, 每行代表一个点的 x 和 y 坐标
- ✓ 使用 `qsort` 函数的关键是定义满足需求的比较函数, 考虑数组内存空间中数据构成和含义:
 - `pt` 中每一行两个 `double` 元素代表一个坐标, 将这2个`double`看作一个单元, 那么数组 `pt` 中有 N 个连续单元
 - 明确比较函数中元素指针指向的内容: 要把2个`double`元素看作一个排序单元, 因此元素指针应该指向 2 个连续的 `double` 元素空间

`double pt[N][2]`

1.1	2.2
1.3	1.3
2.6	4.6
0.2	1.7
...	...

一行两个元素看成一个排序单元



D01-11 : 点集排序

◆使用快速排序

```
#include <stdio.h>
#include <stdlib.h>
#define LINE 6
#define ROW 2
int cmp(const void *d1, const void *d2);
int main()
{
    double pt[LINE][ROW] = {{1.0, 2.0}, {3.0, 4.0}, {2.5, 9.1},
                             {3.5, 3.0}, {3.0, 7.0}, {3.5, 2.0}};
    qsort(pt, LINE, sizeof(pt[0]), cmp); //指定数组一个元素的大小
    //qsort(pt, LINE, 2 * sizeof(double), cmp);
    for (int i = 0; i < LINE; i++)
        for (int j = 0; j < ROW; j++)
            printf(j == 1? "%.1f\n" : "%.1f ", pt[i][j]);
    return 0;
}
```



D01-11: 点集排序

◆编写比较函数

```
int cmp(const void *d1, const void *d2)
{
    double *pt1, *pt2;
    pt1 = (double*)d1; //转换成目标类型
    pt2 = (double*)d2; //转换成目标类型
    if (pt1[0] < pt2[0]) return -1; //0号位置为x坐标，升序
    else if (pt1[0] > pt2[0]) return 1;
    else{
        //x坐标相等，则比较y坐标，降序
        if (pt1[1] < pt2[1]) return 1;
        else if (pt1[1] > pt2[1]) return -1;
        else return 0;
    }
}
```



典型错误案例分析

```
int main()
{
    char *string,c;
    scanf("%s\n", string);
    scanf("%c", &c);
    insert(*string,c);
    return 0;
}
```

```
#include <stdio.h>
char *insert(char *string, char c){
    int i;
    char s[50];
    for(i=0; *string<c; i++)
        s[i]=*string++;
    s[i++]=c;
    for(; *string!='\0'; i++)
        s[i]=*string++;
    return s;
}

int main() {
    char s1[50],c;
    scanf("%s\n",s1);
    scanf("%c",&c);
    printf("%s",insert(s1,c));
}
```

定义了一个局部数组

该循环没有复制
\0。字符数组s没有
\0

不能返回一个局部数组。因为它的生存期为当前函数。



提纲：数据结构程序设计基础（C语言）

- ◆ 1.1 字符串
- ◆ 1.2 二维（多维）数组
- ◆ 1.3 指针
- ◆ 1.4 结构和联合
 - ✓ 1.4.1 结构基础
 - ✓ 1.4.2 结构应用
- ◆ 1.5 文件处理



自定义数据类型：从数据类型说起

- ◆ 大部分高级语言都是强类型语言，这就意味这任何变量都有明确的数据类型
- ◆ 高级语言一般都提供了很多基本数据类型，如C中：
 - ✓ int、char、float、double...
 - ✓ 相应还有指针类型（变量的地址）、数组类型（多个同类型变量作为一个整体）
- ◆ 这些数据类型够用吗？
- ◆ 如何表示日期(Date)？



日期 (Date)

◆可能的选择

- ✓一个整数(int)?
- ✓一个双精度数(double)?
- ✓一个字符串(char*)?
- ✓三个整数，分别表示年、月、日信息
- ✓int year;
- ✓int month;
- ✓int day;

◆存在的问题

- ✓年、月、日之间相互独立，难以体现时间的概念
- ✓对时间进行运算、传递参数时，需要逐个操作，比较复杂
- ✓.....



C语言的解决方案

```
◆ struct date{  
    int year;  
    int month;  
    int day;  
};
```

```
struct 结构类型名 {  
    成员类型 成员名;  
    成员类型 成员名;  
    ...  
};
```

✓ date是结构类型，它是一种新的数据类型

◆ 定义时间变量（结构变量）： **struct date** d1;

◆ 通过"."运算符访问结构变量中的每个成员：

d1.year=2019; d1.month=3; d1.day=3;

✓ 结构体中成员都是普通变量，是日期类型的一部分，称成员变量



关于C语言中的结构类型

◆一种用户自定义的普通数据类型

- ✓可以定义该类型的变量、数组、指针.....
- ✓其成员可以是任意类型
- ✓基本类型、数组、指针、结构.....
- ✓可以做函数的参数类型和返回值类型

◆struct类型的变量

- ✓可以互相赋值
- ✓可以取地址&
- ✓可以作为函数的参数或返回值
- ✓不可以参与运算
- ✓其成员个个也都是如假包换的变量



结构指针

```
◆ struct date{  
    int year;  
    int month;  
    int day;  
};
```

```
◆ struct date d;  
  struct date* pd;
```

✓ pd = &d;

✓ 怎样通过pd访问d的成员?

➤ (*pd).year = 2019;

➤ pd -> year = 2019; /*箭头运算符, 更常用*/



三种结构变量说明

- ◆无结构名，直接声明变量，一般用于说明本地变量

```
struct {  
    ...  
} s1, s2;
```

- ◆有结构名，通常用来说明外部结构变量，或需要在多个函数中用到的相同的结构的变量

```
struct date{  
    ...  
};  
struct date s1, s2; // 必须使用struct（C++中不需要），以说明是结构类型
```

- ◆使用typedef，结构变量定义更简洁

```
typedef struct date{  
    ...  
} Date, *PDate;  
Date d, *pd1, ad[10]; PDate pd2; //不需要struct
```



类型定义关键字: typedef

◆类型定义的语法格式为

typedef 原类型名 新类型名

✓为原类型声明一个新的别名

◆如:

```
typedef int LENGTH;  
typedef char *STRING;
```

◆变量说明为:

```
LENGTH len, maxlen;  
STRING lineptr[LINES], alloc( );
```

✓这与如下直接说明等价:

```
int len, maxlen;  
char *lineptr[LINES], *alloc( );
```

typedef常见用法:

1. 一些安全关键的软件中需要在程序中明确运行环境的数据类型长度, 如:

```
typedef int INT32;  
typedef short INT16  
INT32 port0, port1;  
...
```

2. 用来定义结构类型, 如FILE就是一个用typedef定义的结构类型。

3. 数据结构中用来定义一个链表结点类型:

```
typedef struct node {  
    int n;  
    struct node *next;  
} Node, *Nodeptr;  
Nodeptr list, p;  
实际等价于:  
struct node *list, *p;
```



结构说明可以嵌套

- ◆结构成员可以具有各种类型，当然也可以是其它结构类型，即结构可以嵌套定义
- ◆如下面为描述个人信息（姓名、住址、单位、薪水、生日）的结构说明：

```
struct person
{
    int ID;
    char name[32];
    char address[64];
    char department[64];
    double salary;
    struct date birthdate;
};
struct person table[100];
```

ID	姓名	单位	住址	工资	出生年月		
					年	月	日
1	张三	北航计算机学院	北航家属楼xxx	3000	1980	5	18
...	...	...	...	...	...	...	...



结构说明可以嵌套（续）

- ◆ 结构成员的类型不能是该结构本身，因为它无法确定此结构的边界
- ◆ 但它可以是指向本身结构的指针：自引用结构
 - ✓ 数据结构中会大量使用这种结构体

```
struct keyword
{
    char *name;
    int count;
    struct keyword next; //错误
} *base;
```

```
struct keyword
{
    char *name;
    int count;
    struct keyword *next;
} *base;
```



结构变量的初始化

◆ 结构变量在定义时可以初始化，如：

- ✓ `struct date d = { 1984, 12, 27};`

◆ 结构变量亦可通过整体赋值来初始化，如：

- ✓ `struct date d1 = { 1984, 12, 27};`

- ✓ `struct date d2 = d1;`

◆ 结构指针的赋值

- ✓ `struct date *pd, d;`

- ✓ `pd = &d;` //指向已有的结构变量

- ✓ `pd = (struct date*) malloc(sizeof(struct date));` //指向新分配的结构空间

- ✓ `pd -> year = 2019; free(pd);`



D01-12: 复数结构类型

```
#include <stdio.h>
struct complex {
    double  real;
    double  imag;
};

struct complex addComplex(struct complex c1, struct complex c2);

int main(){
    struct complex c1, c2, c3;
    scanf("%lf%lf%lf%lf", &c1.real, &c1.imag, &c2.real, &c2.imag);
    c3 = addComplex(c1, c2);
    printf("(%.2f, %.2f) + (%.2f, %.2f) = (%.2f, %.2f)\n",
           c1.real, c1.imag, c2.real, c2.imag, c3.real, c3.imag);
    return 0;
}
```

```
//加法操作，结构体作为参数和返回值
struct complex addComplex(struct complex c1,
                           struct complex c2) {
    struct complex tmp;
    tmp.real = c1.real + c2.real;
    tmp.imag = c1.imag + c2.imag;
    return tmp;
}
```



D01-13: 结构成员的引用

```
#include <stdio.h>
struct
{
    int x;
    char *y;
} *p, A[] = {
    1, "for",
    2, "while",
    3, "do_while",
    4, "switch"};
```

```
int main()
{
    int i;
    p = A;
    printf("%d", ++p->x);
    printf("%d", ++(p->x));
    printf("%d", (p++)->x);
    printf("%d", p++->x);
    printf("%c", *p->y);
    printf("%c", *p->y++);
    printf("%c", *(p->y++));
    printf("%c", *p++->y);
    for (i = 0; i < 4; i++)
        printf("\n%d, %s", A[i].x, A[i].y);
}
```

输出结果:

2 3 3 2 d d o _
3, for
2, while
3, _while
4, switch



联合 (union)

- ◆联合是一种复合数据类型，其中的成员共享相同的存储空间
 - ✓可以在不同时间内维持定义它的不同类型和不同长度的对象，以便合理地保存几种类型中的任何一类变量
- ◆定义形式：
 - ✓union 联合名 {分量表} 联合变量名;



示例：联合

```
union v_tag
{
    int ival;
    float fval;
    char *pval;
} uval;
```

```
//联合的使用：
if (utype == INT)
    printf("%d\n", uval.ival);
else if (utype == FLOAT)
    printf("%f\n", uval.fval);
else if (utype == STRING)
    printf("%s\n", uval.pval);
else
    printf("bad type\n");
```

[注意]：使用联合，用法必须一致，即取出的类型必须是最近存入的类型。因此，在使用联合时，要记住当前存于联合中的类型是什么，不允许存入是一种类型，而取出是另一种类型



示例：联合（续）

◆联合可以出现在结构和数组中，数组和结构也可以出现在联合中，下例是在编译中常见到的符号表：

◆例：

```
struct
{
    char *name;
    int flags;
    int utype;
    union
    {
        int ival;
        float fval;
        char *pval;
    } uval;
} symtab[NSYM];
```

```
int main()
{
    int n = 5;
    char c = 'a';
    double pi = 3.14159;
    ...
}
```

name	Flags	uype	uval
n		INT	5
pi		DOUBLE	3.14159
c		CHAR	'a'
...	...	...	...

◆使用：symtab[i].uval.ival



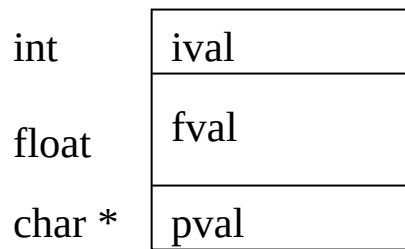
结构和联合的异同

假设定义如下具有相同内容的结构和联合：

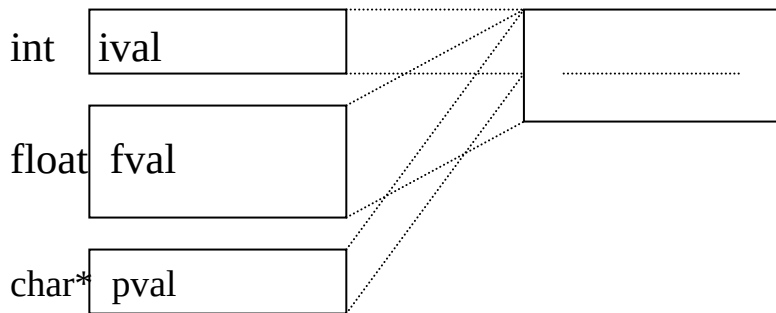
```
struct s_tag
{
    int ival;
    float fval;
    char *pval;
} sval;
```

```
union u_tag
{
    int ival;
    float fval;
    char *pval;
} uval;
```

则struct v_tag和union v_tag的物理存储形式可参见下图：



sval



uval



结构和联合的异同 (续)

◆结构与联合的异同

- ✓在定义或说明形式上，union和struct很类似，但在任何时刻联合只允许联合中说明的某一成员留在联合里
- ✓结构由多个成员（分量）所组成，而联合只有一个成员，只不过该成员的名字和类型可以在规定的几个里选定一个

◆联合可以看作是一个特殊的结构，其所有成员在结构中的位移量都是0，当对联合变量分配存储空间时，应保证它能容纳最大的一个成员的大小，而存储空间的边界应能适于联合中的所有可选成员的类型

- ◆对联合的初始化只能对应于它的第一个可选成员
- ◆对联合成员的引用方式完全和结构的情况一样



D01-14综合应用：通信录整理

◆【问题描述】

- ✓读取一组电话号码簿（由姓名和手机号码组成），将重复出现的项删除（姓名和电话号码都相同的项为重复项，只保留第一次出现的项），并对姓名相同手机号码不同的项进行如下整理：首次出现的项不作处理，第一次重复的姓名后面加英文下划线字符_和数字1，第二次重复的姓名后面加英文下划线字符_和数字2，依次类推。最后对整理后的电话号码簿按照姓名进行从小到大排序，并输出排序后的电话号码簿

◆【输入形式】

- ✓先从标准输入读取电话号码个数，然后分行输入姓名和电话号码

◆【输出形式】

- ✓按照姓名从小到大的顺序分行输出最终的排序结果，先输出姓名再输出电话号码，以一个空格分隔



D01-14: 通信录整理

◆ 【输入样例】

15

liping 13512345678

zhaohong 13838929457

qiansan 13900223399

zhouhao 18578294857

anhai 13573948758

liping 13512345678

zhaohong 13588339922

liping 13833220099

boliang 15033778877

zhaohong 13838922222

tianyang 18987283746

sunnan 13599882764

zhaohong 13099228475

liushifeng 13874763899

caibiao 13923567890

□ 【输出样例】

anhai 13573948758

boliang 15033778877

caibiao 13923567890

liping 13512345678

liping_1 13833220099

liushifeng 13874763899

qiansan 13900223399

sunnan 13599882764

tianyang 18987283746

zhaohong 13838929457

zhaohong_1 13588339922

zhaohong_2 13838922222

zhaohong_3 13099228475

zhouhao 18578294857



问题分析

◆几个关键点

✓1. 去重（名称和电话号码相同）

- 简单方法：输入时直接检查是否已经存在，如果存在，则不保存
- 复杂方法：先全部输入、后排序（名称、电话号码和输入顺序）、再去重

✓2. 重命名（名称相同、电话号码不同）

- 简单方法：从第1个开始，和后面的所有记录依次比较，重复则重命名
- 复杂方法：先排序（名称、输入顺序），再命名（采用好的排序算法，效率相对较高）

✓3. 输出

- 按照调整后的名称从小到大排序



参考代码：总体逻辑

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define LEN 105
//输入电话号码簿，返回读入的数量
int input(Person p[]);
//重命名姓名重复的人员
void reName(Person p[], int n);
//按照名字从小到大的比较函数
int cmp(const void* p1, const void* p2);
int main(){
    Person persons[LEN];
    int i, n;
    n = input(persons);
    reName(persons, n);
    qsort(persons, n, sizeof(persons[0]), cmp);
    for (i = 0; i < n; i++) {
        printf("%s %s\n", persons[i].name, persons[i].phone);
    }
    return 0;
}
```

```
//定义结构体，存储数据
typedef struct _Person
{
    int seq; //输入序号，本题目目前解法无用
    char name[25]; //姓名
    char phone[15]; //电话号码
} Person, * PPerson;
```



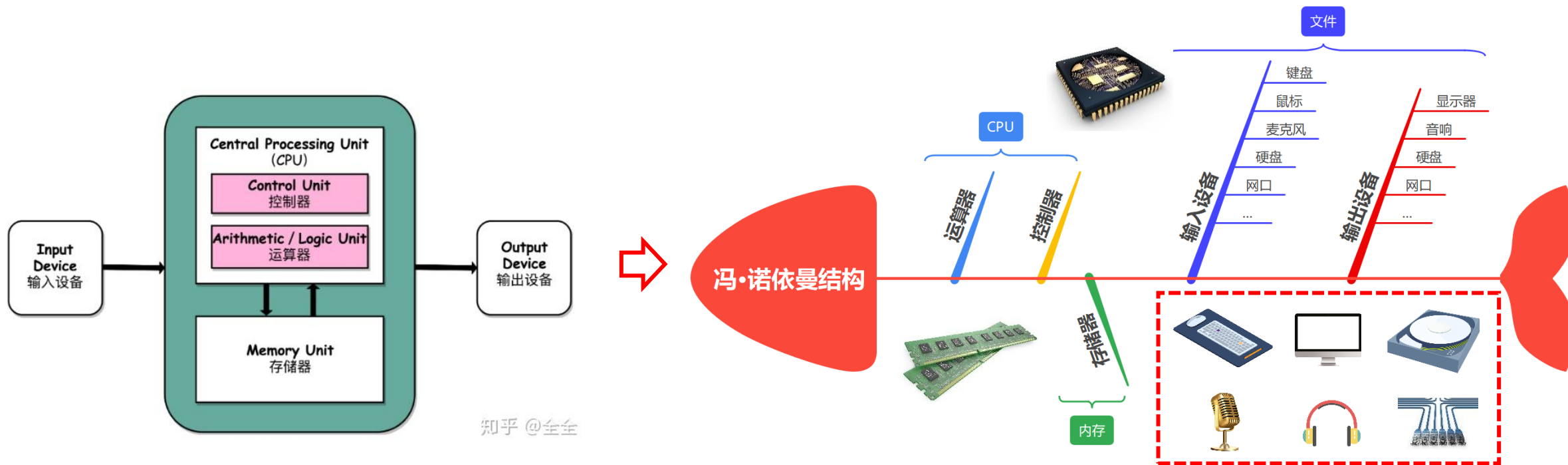
提纲：数据结构程序设计基础（C语言）

- ◆ 1.1 字符串
- ◆ 1.2 二维（多维）数组
- ◆ 1.3 指针
- ◆ 1.4 结构和联合
- ◆ 1.5 文件处理
 - ✓ 1.5.1 文件基本概念
 - ✓ 1.5.2 文件操作流程
 - ✓ 1.5.3 文本文件读写



1.5.1 文件的基本概念

- ◆ 程序运行在内存中，程序本身是以文件的形式长期保存在硬盘中的
 - ✓ 当前程序中所有输入和输出都是临时的，无法长期保存，每次重新输入和输出



知乎 @全全

冯·诺依曼结构计算机的工作原理 (IPO)
计算 + 存储 (+ 外部输入输出)

现代计算机的硬件组成
存在各类不同的外部输入输出设备



文件

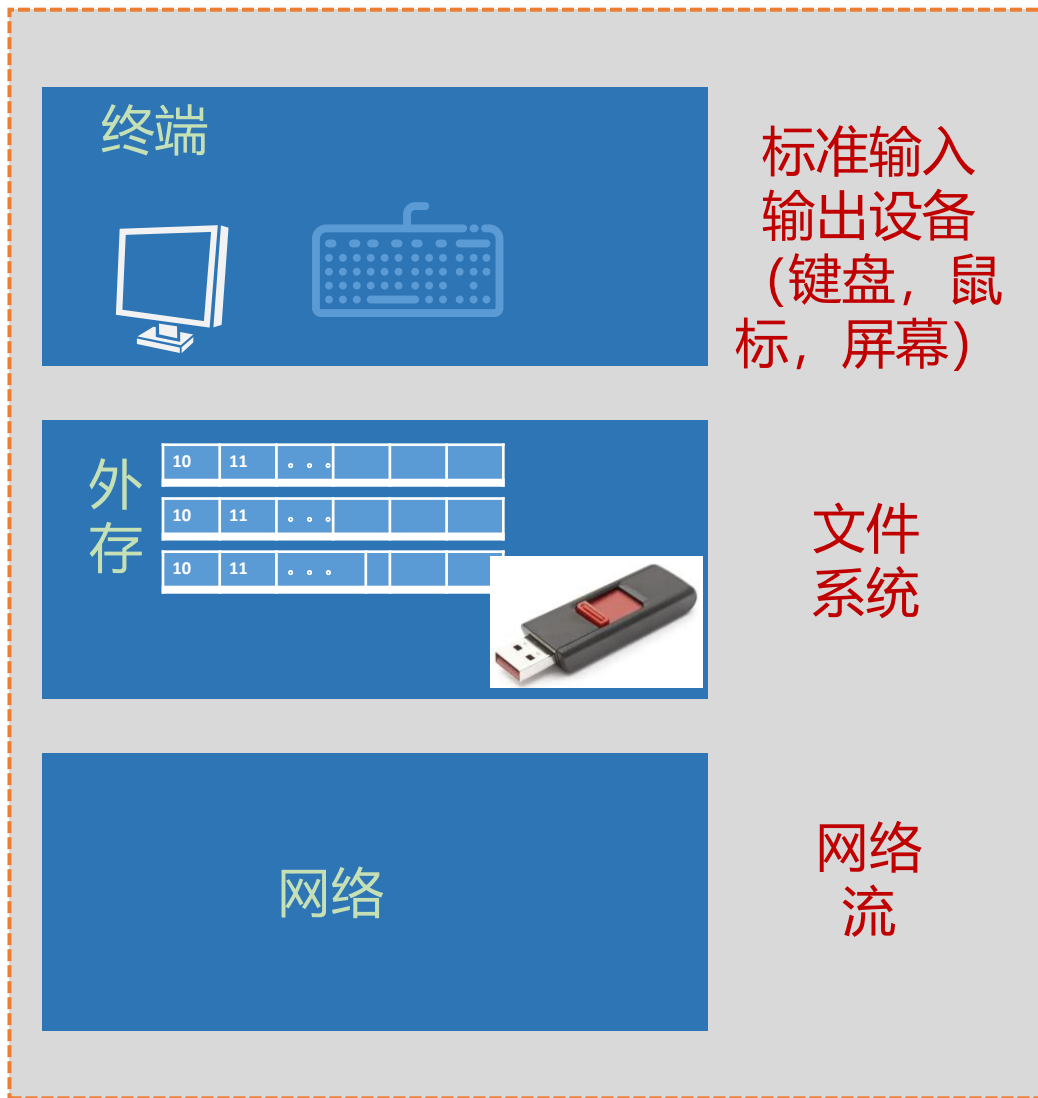
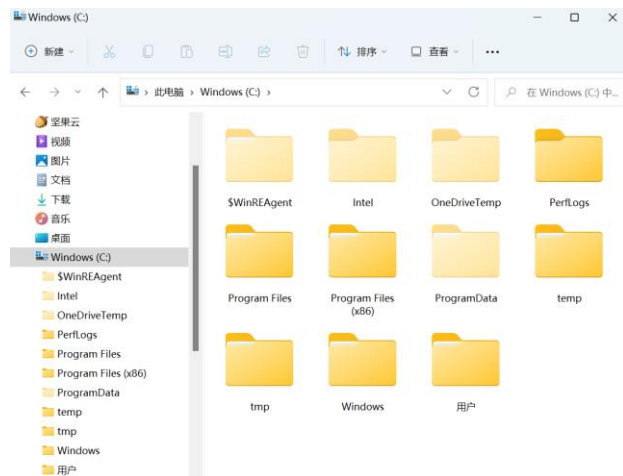
- ◆文件管理：操作系统把“计算机CPU”和“存储器（内存）”之外的外部输入设备统一作为文件来管理，用来长期保存外部数据
- ◆文件是操作系统提供的将数据与硬盘等外部存储设备关联的一种技术
- ◆目前接触到的文件
 - ✓1、自己编写的C程序源文件；
 - ✓2、编程用IDE：CodeBlock、Dev-C等文件
 - ✓3、scanf中的标准输入stdin（键盘）
 - ✓4、printf中的标准输出stdout（屏幕）





文件

- ◆狭义的文件：存储在外部介质（硬盘、U盘、DVD等）上的数据有命名的集合，是操作系统数据管理的单位
- ◆广义的文件：所有指向计算机和存储器之外的任何外部设备
 - ✓操作系统管理的所有输入输出设备





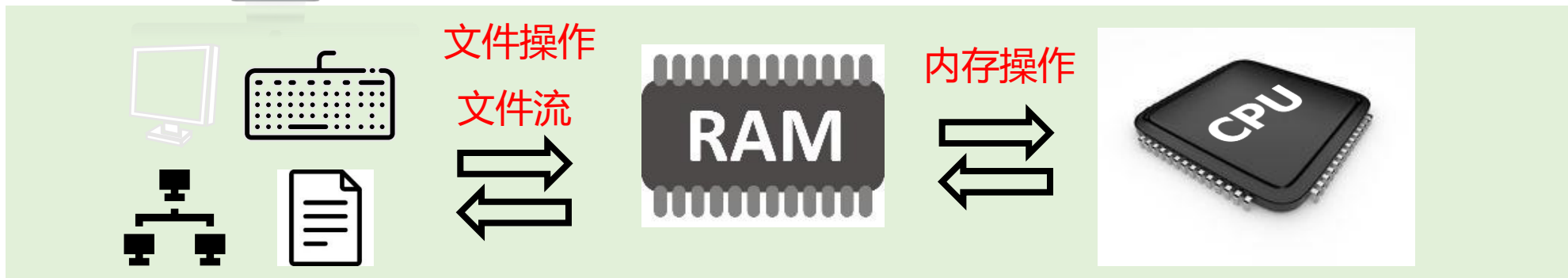
文件访问

- ◆ 文件都要**读入**内存才能处理，数据必须**写入**文件才不会丢失
- ◆ 数据在文件和内存之间传递的过程叫做**文件流**，类似水从一个地方流动到另一个地方
 - ✓ 数据从**文件读入到内存**的过程叫做**输入流**
 - ✓ 从**内存写入到文件**的过程叫做**输出流**



Word, txt,
.c源文件,
音乐, 电影...

- 可打开&关闭
- 可输入/输出
- 读写和运行速度不对等



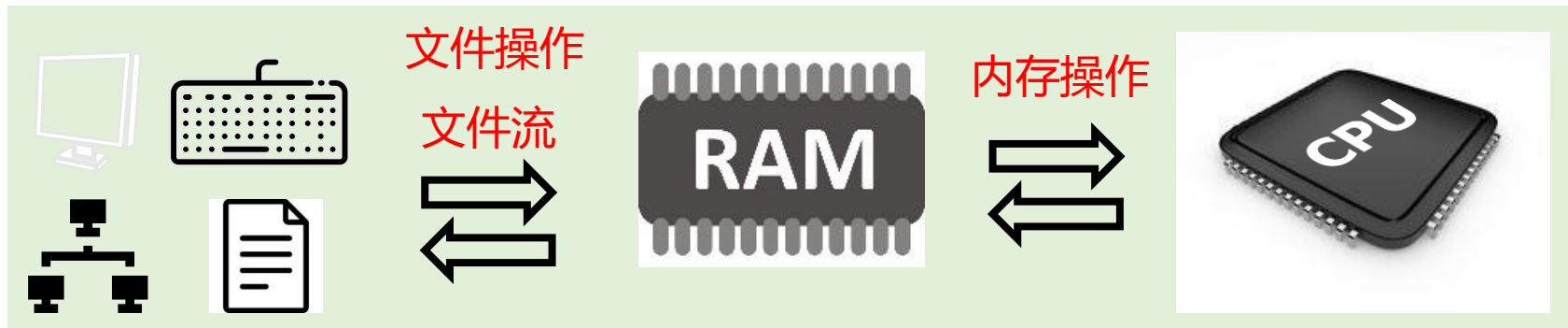
文件流

- 可打开&关闭
- 可输入/输出
- 读写速度不对等
- 点到点
- 批量传输
- 有限速度
- 队列顺序

操作方式

- 整体操作
 - 空间问题
- 逐个操作
 - 时间问题
- 逐批操作
 - 需要缓存
 - 双向读写
 - 双份地址

文件操作很复杂!



内存的存取速度远高于硬盘的存取速度，而CPU内高速存储器的存取速度更是远高于内存的存取速度

- 程序从硬盘放到内存以后，CPU就直接在内存运行程序，这样比CPU直接在硬盘运行程序就要快很多
- 内存解决了一部分CPU运行过快，而硬盘数据存取太慢的问题。提高了电脑的运行速度



文件分类

◆按存储介质

- ✓普通文件：存储介质文件（磁带、磁盘、光盘、U盘等）
- ✓设备文件：非存储介质（键盘、显示器、打印机等）



机械硬盘内部结构



SSD固态硬盘内部结构



◆按数据的组织形式

✓文本（正文）文件

- 按照字符编码规则，存储字符串形式，方便阅读
- 如：ASCII文件，每个字节存放一个字符的ASCII码

✓二进制文件

- 数据按其在内存中的存储形式原样存放

123456789

文本文件：占9字节

二进制文件：占4字节（int）

D0	CF	11	E0	A1	B1	1A	E1	00
00	00	00	00	00	00	00	00	3E
06	00	00	00	00	00	00	00	00
E1	00	00	00	00	00	00	00	00
01	00	00	00	FE	FF	FF	FF	00
E0	00	00	00	FF	FF	FF	FF	FF
FF	FF	FF	FF	FF	FF	FF	FF	FF



设备文件

- ◆在操作系统中，为了统一对各种硬件的操作，简化接口，不同的硬件设备也都被看成一个文件
 - ✓对这些文件的操作，等同于对磁盘上普通文件的操作
- ◆C程序开始运行时有三个自动打开的标准文件：
 - ✓标准输出文件 (**stdout**)：通常指显示器，printf 就是向这个文件输出数据；
 - ✓标准输入文件 (**stdin**)：通常指键盘，scanf 就是从这个文件读取数据；
 - ✓错误信息输出文件 (**stderr**)：通常指显示器，输出错误信息。



Word, txt, .C源文件, 电影.....





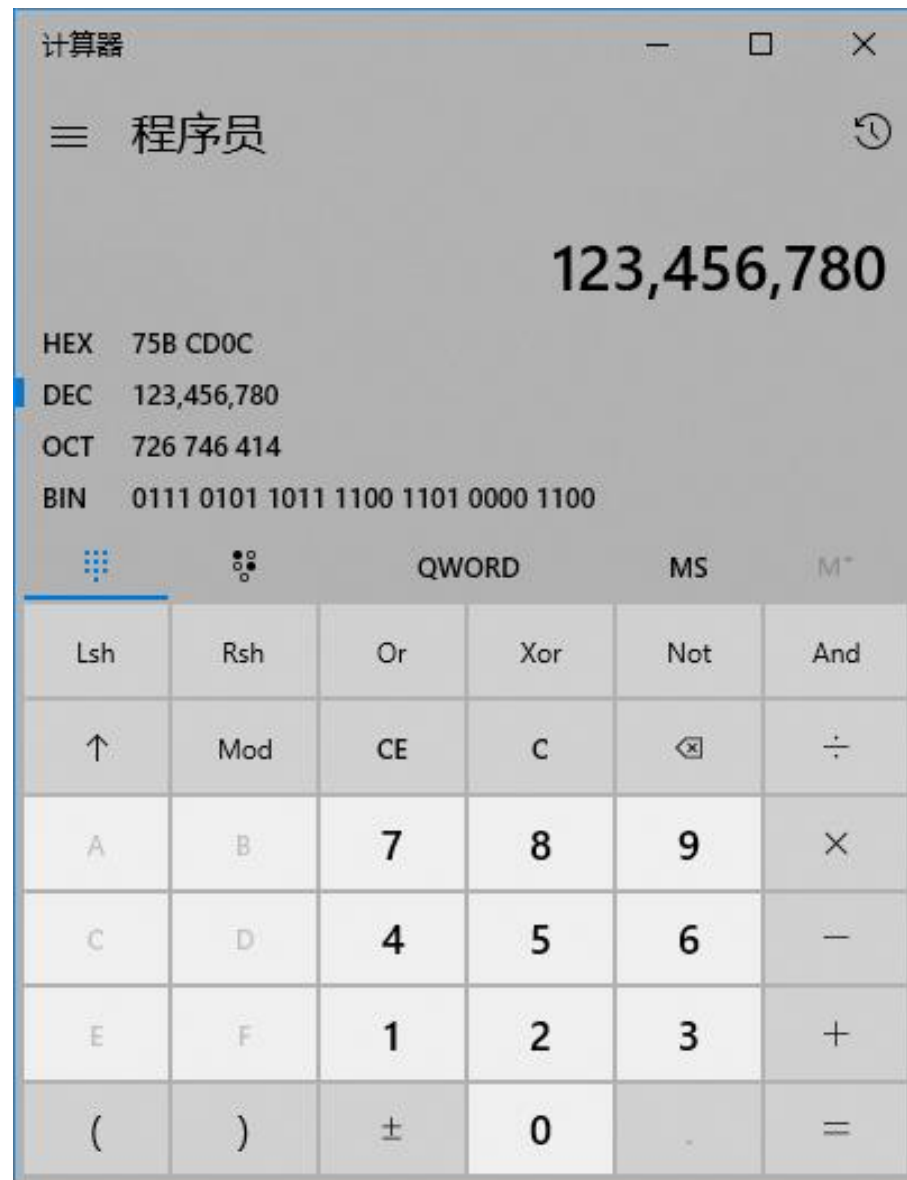
文本文件与二进制文件

◆ 文本文件，又称正文文件

- ✓ 对字节按照某种字符集（如ASCII）编码
- ✓ 如：123456780占9字节
- ✓ 不同操作系统中对正文文件的规定格式有所不同

◆ 二进制文件

- ✓ 数据按其在内存中的存储形式原样存放，
- ✓ 若一个int占4个字节，则123456780占4字节，
存储为：
- ✓ 0000 0111 0101 1011 1100 1101 0000 1100
- ✓ 十六进制表示为：07 5B CD 0C





文本文件与二进制文件

C把文件看成一系列连续的字节，每个字节都能单独读取。这种模式是从C发源地UNIX操作系统来的。别的操作系统可能无法跟这个模型完全一致，所以C提供了两种文件的模式。

正文文件

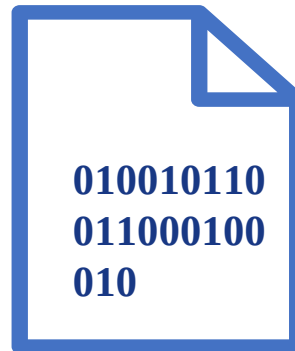
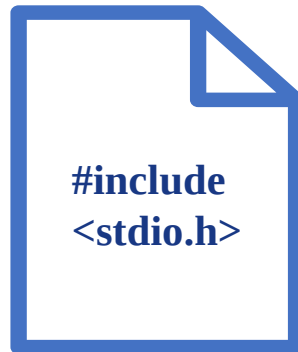
常见有：.C文件，.h文件，.txt文件等可直接阅读

文本编码

ASCII编码

Unicode编码

文本模式会映射成统一的C模式。读写内容与文件的实际内容不一致！

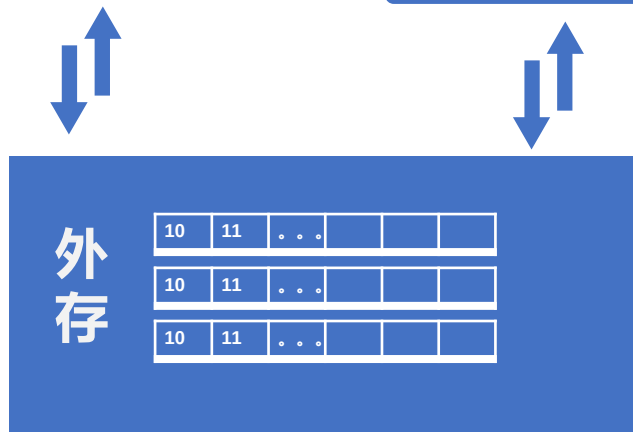


二进制文件

常见有：.obj文件，.exe文件，图片、音频、视频等，需要相关程序处理才能显示

任意编码（跟实际文件格式相关）

二进制文件在内存中与文件中是完全一致的！



所有文件内容都以二进制存储



不同操作系统文本文件格式不一致

自动转换

↑ **读出**
\r\n转换为**\n**
^Z解释为文件结尾
↓ **写入**
\n转换为**\r\n**

```
#include <stdio.h>
int main()
//Your C code here
```

如何描述正文行结束

如何描述文件结束

写入 读出

Windows

```
#include <stdio.h> \r\n
int main() \r\n
This is a C code \r\n
[ ^Z ] (早期版本有, 后来取消了)
```

文本行结束 回车符 ('r', 0x0d)
 换行符 ('n', 0x0a)

文件结束(EOF) ^Z (0x1a)

文件系统
(磁盘)

Unix/Linux

```
#include <stdio.h> \n
int main() \n
This is a C code \n
```

文本行结束 换行符 ('n', 0x0a)

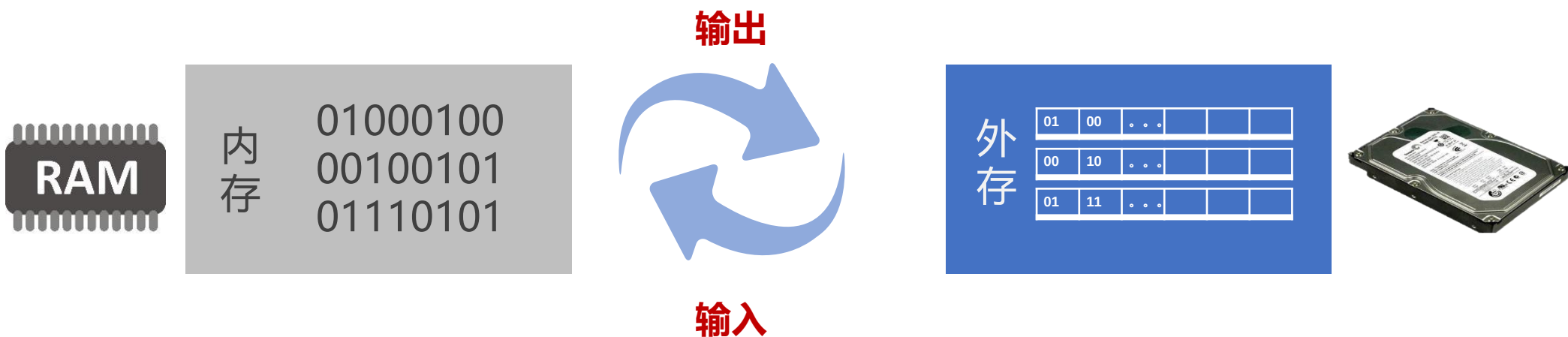
文件结束(EOF) ^D

无法采用内存与文件完全一致的模型，所以分为文本模式和二进制模式



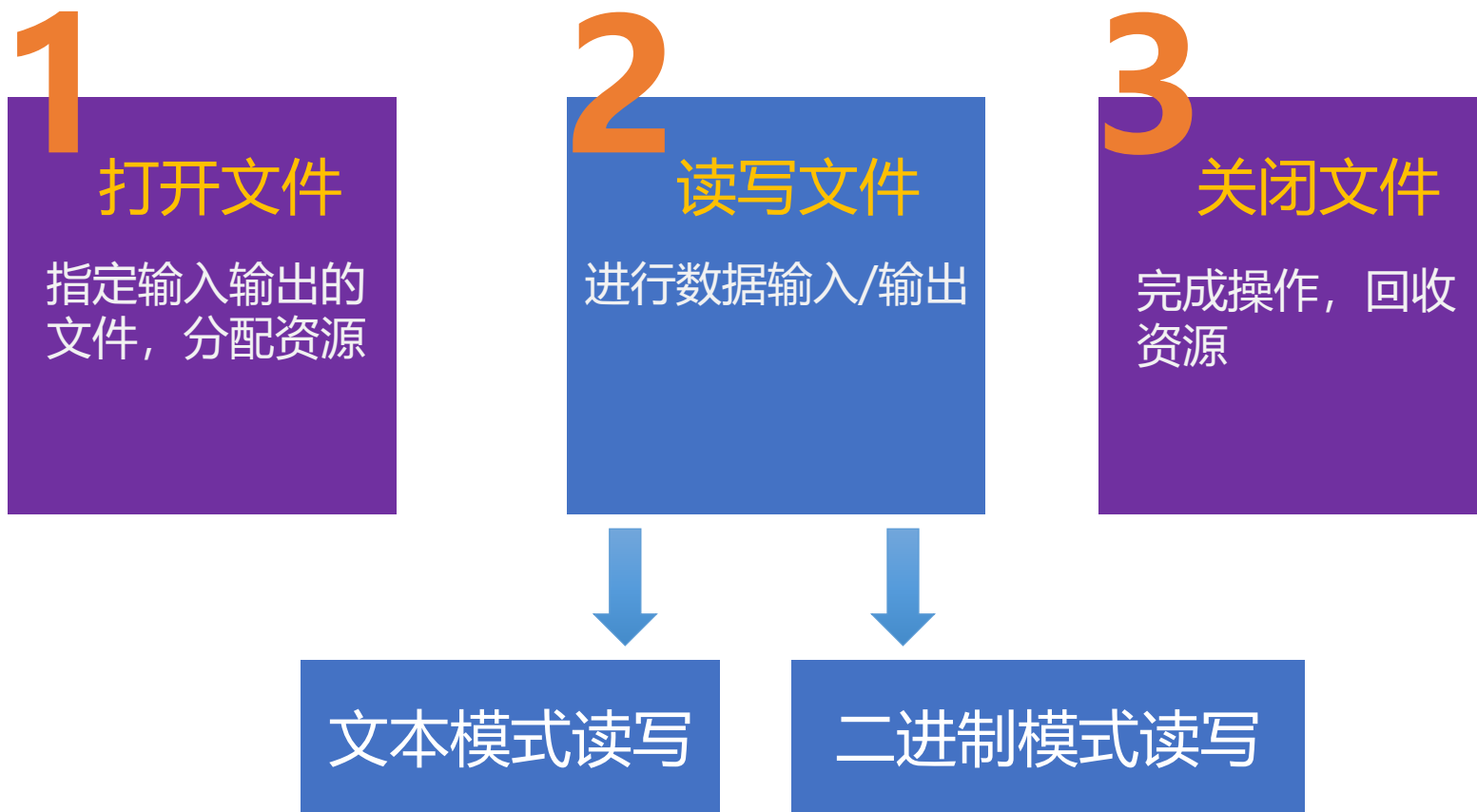
二进制模式

- ◆ 不对文件进行任何解释
- ◆ 不在数据流中添加任何其他字符
- ◆ 数据在程序内存中和在文件中是完全一致的





1.5.2 文件操作流程

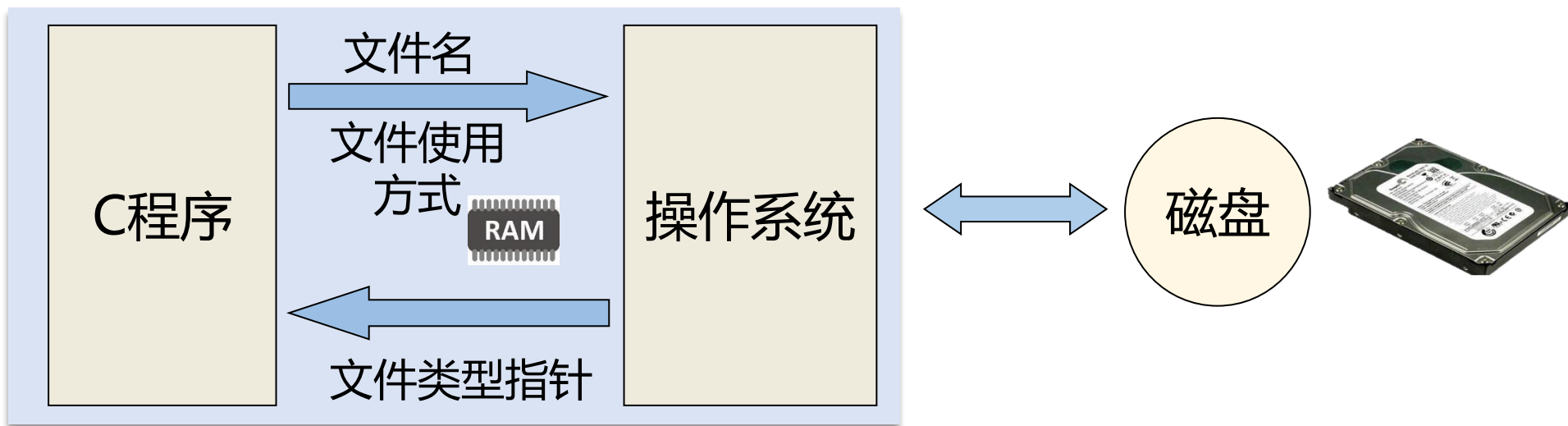




C语言中的文件操作

◆文件指针变量： `FILE *fp;`

- ✓文件操作时，关键概念是指向FILE类型的“文件类型指针”，简称文件指针
- ✓FILE是C语言定义的一个结构体，会记录文件的基本信息
- ✓文件打开时，系统自动建立文件结构体，并把指向它的指针返回来，程序通过这个指针获得文件信息，访问文件
- ✓文件关闭后，它的文件结构体被释放





文件类型指针

◆文件结构体FILE的指针

- ✓缓冲文件系统为每个正使用的文件在内存开辟文件1
- ✓文件信息用系统定义的名为FILE的结构体描述



◆FILE定义在stdio.h中

```
typedef struct _iobuf
{
    char *_ptr;
    int _cnt;
    char *_base;
    int _flag;
    int _file;
    int _charbuf;
    int _bufsiz;
    char *_tmpfname;
} FILE;
```

文件结构中包括：文件名、文件编号、缓冲区中剩下的字符数、文件操作方式、文件当前读写位置、文件缓冲区位置、等等



打开文件：获取文件指针-参数1：路径

```
FILE *fopen(const char *path, const char *mode);
```

path是一个字符串，指定需要打开的文件的路径名

D:\CC\temp\file.dat

文件路径

文件名主干

后缀

```
int main()
{
    FILE *fp;
    char ch, *filename = "c:\\cc\\file.c";
    if ((fp = fopen(filename, "r")) == NULL)
    {
        printf("cannot open file\n");
        return 0;
    }
    ...
}
```

UNIX/Linux `"/Home/yin/cprog/file.c"` 、 `"./test.c"` 、 `../dir/test.c"` 、 `"doc"`

Windows `"c:\\Windows\\system32abcv.dll"`、 `"\\.\\debug\\data.txt"`

windows平台，反斜线\在c语言中作为转义引导字符使用，因此带目录名的路径必需双写反斜线符。另外，需要注意在windows上文件名不区分大小写，而在UNIX/Linux平台中，是严格区分的
目录路径中：. 代表当前目录，.. 代表上一级目录，默认为当前目录！



打开文件：获取文件指针-参数2：打开模式

```
FILE *fopen(const char *path, const char *mode);
```

mode是一个字符串，指定打开文件的方式

```
char *filename = "c:\\cc\\file.c";  
if((fp = fopen(filename, "r")) == NULL)
```

字符串	含义	模式
"r"	读方式，当文件不存在时打开失败	文本模式
"w"	写方式，当文件存在时内容被清空	
"a"	追加方式，将数据写到文件末尾，当文件不存在时创建该文件	
"r+"	读写方式，读写位置在文件开头处	
"w+"	读写方式，当文件存在时清空该文件	
"a+"	读和追加方式，将数据写到文件末尾，当文件不存在时创建该文件	二进制模式
"rb"、"wb"、"ab"、"ab+"、 "a+b"、"wb+"、"ab+"、"a+b"	跟上面的方式类似，但是以二进制模式而不是正文模式打开文件	

Unix/Linux平台上，带字母b的模式和不带字母b的模式相同



打开文件：获取文件指针

```
FILE *fopen(const char *path, const char *mode);
```

- ◆ fopen 的返回值是一个 **FILE *** 类型的指针，指向被打开的文件。
- ◆ 随后的读写操作中，都需要使用这一指针说明所操作的文件。
- ◆ 如果不成功，则返回**NULL**。

```
// 【例】打开文件 检测打开文件时的错误
#include <stdio.h>
int main(){
    FILE *fp;
    fp = fopen("file_a.txt", "r");
    if (fp == NULL)
    {
        perror("Can't open file_a");
        return 1;
    }
    ...
}
```

需要报告错误原因时，可用perror()函数。

```
void perror(const char *string);
```

如果文件不存在，这段代码输出结果如下：

```
Can't open file_a: No such file or
directory
```



关闭文件

```
int fclose(FILE *fp);
```

- ◆ **参数**: fp 是打开文件时得到的指针, 指向被打开的文件
- ◆ **功能**: 关闭fp指向的文件, 使文件指针与文件“脱钩”, 释放文件结构体和文件指针
- ◆ **返回值**: 正常关闭为0; 出错时,非0
- ◆ **不关闭文件可能会丢失数据**
 - ✓ 一般文件操作函数都是有缓冲的, 对文件内容的操作是在内存中进行的, 不一定及时写入硬盘, 只有关闭文件后才确保正确地写入到硬盘中



文件操作函数（一）

函数原型	函数功能
<code>FILE *fopen(const char *path, const char *mode);</code>	打开由path指定的文件，成功时返回字符流指针，否则返回NULL。此后对该指定文件的读写均通过本函数返回的字符流指针进行
<code>int fclose(FILE *fp);</code>	关闭由字符流指针fp指向的文件。成功时返回0，否则返回EOF
<code>int fread(void *buf, size_t size, size_t num, FILE *fp);</code>	从字符流指针fp中将num个大小为size字节的数据项读入缓冲区buf中
<code>int fwrite(const void *buf, size_t size, size_t num, FILE *fp);</code>	将num个大小为size字节的数据项从缓冲区buf中写到fp指向的字符流中
<code>int fseek(FILE *fp, long offset, int whence);</code>	将字符流fp的读写位置修改为由whence起算的offset处。成功时返回0，否则返回-1
<code>long ftell(FILE *fp);</code>	返回字符流fp的当前读写位置
<code>int fflush(FILE *fp);</code>	强制将暂存在缓冲区的数据写入字符流fp中
<code>int fileno(FILE *fp);</code>	获取字符流fp所对应的文字描述字
<code>int feof(FILE *fp);</code>	检查字符流fp的结尾标记是否被设置



文件操作函数（二）

函数原型	函数功能
<code>int ferror(FILE* fp)</code>	检查字符流fp的操作错误标记是否被设置
<code>void clearerr(FILE* fp)</code>	清除字符流fp的文件结尾标记和操作错误标记
<code>int fgetc(FILE* fp)</code>	从字符流fp中读取一个字符
<code>int fputc(int c, FILE* fp)</code>	向字符流fp中写入一个字符
<code>char* fgets(char* s, int size, FILE* fp)</code>	从字符流fp中读入一行不超过size个字符并保存在s指定的缓冲区中
<code>int fputs(const char* s, FILE* fp)</code>	将字符串s写入字符流fp中
<code>int fscanf(FILE* fp, const char* format,...)</code>	按format指定的格式从字符流fp中读入数据
<code>int fprintf(FILE* fp, const char* format, ...)</code>	向字符流fp中按format指定的格式写出数据



标准输入输出是一类特殊的文件

1

打开文件

2

读写文件

3

关闭文件

```
int printf(const char *format,...);  
int scanf(const char *format,...);
```

- ◆每个C程序运行时，至少需要一些输入输出设备才能正常运行。所以系统**自动打开和关闭**三个标准设备，也即标准文件：
 - ✓标准输入 stdin
 - ✓标准输出 stdout
 - ✓错误信息输出 stderr
- ◆对标准文件输入输出函数省略了对这些文件的说明和打开关闭操作



1.5.3 文本文件读写：对照标准输入输出操作

功能	对标准输入输出设备的I/O函数	对文本文件操作的I/O函数
单个字符读写	int getchar(void)	int fgetc(FILE *fp), int getc(FILE *fp)
	int putchar(int c)	int fputc(int c, FILE *fp), int putc(int c, FILE *fp)
格式化读写	int printf(const char *format,...)	int fprintf(FILE *fp, const char *format,...)
	int scanf(const char *format,...)	int fscanf(FILE *fp, const char *format,...)
字符串读写	char *gets(char *s)	char * fgets(char *s, int n, FILE *fp)
	int puts(const char *s)	int fputs(const char *s, FILE *fp)

- ◆与标准输入输出函数相比，文本文件的操作函数一般多一个文件指针的参数，其他**使用方式基本相同**（个别函数实现有差异）
- ◆同理，文件操作函数也可以直接用于标准输入输出，只需要把文件指针换成标准输入输出设备

```
printf("Hello\n") <==> fprintf(stdout, "Hello\n")  
getchar() <==> fgetc(stdin)
```




(1) 单字节（字符）读写：fputc和fgetc

◆单个字符I/O: fputc与fgetc

✓读取或写入的字节变量一般建议用int类型！

➤https://blog.csdn.net/little_bobo/article/details/55189472

//作用：将一个字节代码**c**输出到**fp**所指向的文件中，通常用于往文本文件中写入一个字符**c**
//返回值：输出成功则返回所输出的字符**c**，失败则返回**EOF**

```
int fputc(int c, FILE *fp)
```

//作用：从**fp**所指向的文件中读入一个字节代码，通常用于从文本文件**fp**中读取一个字符
//返回值：返回所读的代码值，如发生错误或读到文件结束符则返回**EOF**

```
int fgetc(FILE *fp)
```



D01-15: 文件内容拷贝

◆输入源文件和目标文件， 将源文件内容拷贝到目标 文件中

- ✓源文件（读）**必须存在**，
否则报错
- ✓目标文件（写）可以**存在**
可以不存在，如果存在，
则**清空**写入新内容；如果
不存在，则创建新文件，
再写入

```
#include <stdio.h>
int main(){
    FILE *in, *out;
    char infile[20], outfile[20]; //源文件和目标文件
    int ch;
    scanf("%s", infile); // 输入源文件
    scanf("%s", outfile); // 输入目标文件
    in = fopen(infile, "rb"); //打开输入文件，二进制方式！
    out = fopen(outfile, "wb"); //打开输出文件，二进制方式
    if (in == NULL) {
        printf("Cannot open infile.\n");
        return 0;
    }
    if (out == NULL) {
        printf("Cannot open outfile.\n");
        return 0;
    }
    while ((ch = fgetc(in)) != EOF)
        fputc(ch, out);
    fclose(in);
    fclose(out);
}
```



(2) 格式化读写: fprintf和fscanf

◆ 格式化I/O: fprintf与fscanf

✓ 类似于printf和scanf, sprintf和sscanf的区别

//作用: 按格式对文件进行I/O操作

//返回值: 成功, 返回I/O的个数; 出错或文件尾, 返回EOF

```
int fprintf(FILE *fp, const char *format[,argument,...])
```

```
int fscanf(FILE *fp, const char *format[,address,...])
```

//示例: 使用格式化文件读写函数

```
fprintf(fp, "%d, %6.2f", i, t); // 将i和t按%d, %6.2f格式输出到fp文件
```

```
fscanf(fp, "%d, %f", &i, &t); // 若文件中有3, 4.5, 则将3送入i, 4.5送入t
```

```
fprintf(stdout, "%d, %6.2f", i, t); //即: printf("%d, %6.2f", i, t)
```



D01-16: 表达式计算 (读文件并计算)

- ◆【问题描述】文件“expr.txt”中每一行都存储了一个表达式，包括两个运算数（整数）和一个运算符（+、-、*、/）（两个运算数以运算符分隔，中间没有其它任何字符），请读出来，并计算该行结果，依次输出结果
- ◆【输入形式】无
- ◆【输出形式】在屏幕上逐行输出表达式即计算结果（说明：除法按照整数计算）
- ◆【样例输入】“expr.txt”文件内容如下：
✓ 1+1
6-2
22*3
23/4
- ◆【样例输出】屏幕输出为：
✓ 1+1=2
6-2=4
22*3=66
23/4=5



读文件并计算

```
int main(){
    FILE *fp;
    int num1, num2;
    char op;
    fp = fopen("expr.txt", "r");
    if(fp==NULL){
        printf("Can't open the file!\n");
        return -1;
    }
    while(fscanf(fp, "%d%c%d", &num1, &op, &num2) != EOF){
        printf("%d%c%d=%d\n",
            num1, op, num2, calc(num1, op, num2));
    }
    fclose(fp);
    return 0;
}
```

```
int calc(int n1, char op, int n2){
    switch(op){
        case '+': return n1+n2;
        case '-': return n1-n2;
        case '*': return n1*n2;
        case '/': return n1/n2;
        default: return 0;
    }
}
```

◆与标准控制台输入的不同之处:

- ✓ (1) 定义文件类型的变量
- ✓ (2) 通过fopen函数, 打开文件, 将变量与硬盘上的文件关联
- ✓ (3) 使用f系列的函数读写文件 (多一个FILE类型参数)
- ✓ (4) 通过fclose函数, 断开变量与文件的关联, 关闭文件



(3) 字符串读写: fgets与fputs

◆字符串I/O: fgets与fputs

```
//从fp所指文件读最多n-1个字符送入s指向的内存区，并在最后加一个 '\0'  
//若读入n-1个字符前遇换行符或文件尾（EOF）即提前结束读入  
//正常时返回读取字符串的首地址；出错或文件尾，返回NULL  
char *fgets(char *s, int n, FILE *fp);
```

```
//把s指向的字符串写入fp指向的文件  
//正常时返回写入的最后一个字符；出错为EOF  
int fputs(char *s, FILE *fp);
```

◆fgets与gets的不同

- ✓ fgets多一个参数n，**最多**只读入n-1个字符（\0占一个）
- ✓ fgets**原封不动**得读取文件的内容，文件中的**\r\n会读入字符串**，并再最后补充\0，而gets会自动将\n转换为\0

◆fputs与puts的不同

- ✓ Fputs原封不动的输出字符串，并不补充回车



D01-17: 输出文件中

◆输出文件中最长行（若有多个，输出第一个）的行号、本行全部内容、字符个数

注意：本程序输出中**字符串长度多1**，因为对fgets(s, n, fp)，当n比s的实际长度多2个字符以上时，**fgets会把'\n'也读入，再读入'\0'**，strlen(s)就包括了'\n'（注意可能还有'\r'，取决于所用的操作系统）。

```
#include <stdio.h>
#define MAX_N 100
int main(){
    FILE *fp;
    char arr[2][MAX_N] = {{""}}, filename[MAX_N];
    int in = 1, longest = 0, linecount = 0;
    int max_len = 0, len, tmp, lineNum;
    scanf("%s", filename);
    if ((fp = fopen(filename, "r")) == NULL) {
        perror("can not open file\n"); exit(0);
    }
    while (fgets(arr[in], MAX_N, fp) != NULL){
        linecount++; //统计行号
        len = strlen(arr[in]);
        if (len > max_len) {
            lineNum = linecount; // 记录当前最长行
            max_len = len;
            tmp = in; in = longest; longest = tmp;
        }
    }
    printf("the %d\'s line:\n%s\n%d Byte.\n",
        lineNum, arr[longest], max_len);
    fclose(fp);
}
```



使用freopen将文件重定向到标准输入输出

- ◆ 普通文件的读写主要涉及到需要操作FILE类型的变量
 - ✓ 标准输入设备stdin、标准输出设备stdout也是FILE类型的变量
- ◆ C语言提供了重定向函数freopen，可以将某个文件关联到标准输入和输出设备上，从而采用标准输入输出函数操作指定的文件

```
FILE *freopen( const char *filename, const char *mode, FILE *stream );
```




使用freopen重定向标准输入设备

- ◆重定向后，后续标准输入输出设备为重定向后的文件
- ◆D01-16：表达式计算（freopen实现）

```
int main(){
    int num1, num2;
    char op;
    if(freopen("expr.txt", "r", stdin) == NULL){
        perror("Can't open the file!\n");
        return 1;
    }
    while(scanf("%d%c%d", &num1, &op, &num2) != EOF){
        printf("%d%c%d=%d\n",
            num1, op, num2, calc(num1, op, num2));
    }
    fclose(stdin);
    return 0;
}
```



D01-17综合应用：单词计数

◆ 【问题描述】

- ✓读取一个文本文件，统计改文本文件中单词出现的次数，并按照次数从大到小依次输出，如果次数相同，则按单词字典序从小到大输出

◆ 【输入形式】

- ✓文本文件名

◆ 【输出形式】

- ✓单词 出现次数

◆ 【约定】

- ✓文本文件中只包含ASCII字符，任何字母序列均可以认为是单词，任何非字母字符作为单词的分隔符。
- ✓单词不区分大小写，所有的大写字母均统一转换为小写输出
- ✓单词长度不超过50，数量不超过10000



算法设计：主要知识点分析

◆读取文本文件中的单词

- ✓文件操作函数：fopen、fclose
- ✓选择合适的读取函数：fgets、fscanf、fgetc?
- ✓分隔单词：查询非字母分隔符、大写转小写

◆存储和查询单词和出现的次数

- ✓存储：结构体数组（存储位置：有序/无序？）
- ✓查询：单词是否已经存在（顺序查询、折半查询）

◆字典序输出

- ✓无序插入，最后统一排序？
- ✓有序插入（插入单词时保持有序）？



算法设计：伪代码

声明单词表结构体

```
typedef struct _WordTable{  
    char word[50];  
    int count;  
}WordTable, *PWordTable;
```

打开文件;

while (读取到一个单词)

```
int getWord(FILE *fp, char word[]);
```

功能说明：从文件中读入一个有效的单词（小写形式），存入word；如无有效单词，则返回0，否则返回1

if (在已有单词数组中查找该单词)

相应单词次数 + 1;

```
int searchWord(char word[]);
```

功能说明：查询单词表（全局变量）中是否存在单词word，返回位置下标，如果没找到则返回-1

else

插入新单词，次数为1，单词总数+1

```
int insertWord(char word[]);
```

功能说明：在单词表（全局变量）的最后位置，插入单词word，返回插入位置，如果插入失败，则返回-1

按要求对单词表排序;

输出单词及出现次数;

可以使用快速排序库函数，写比较函数即可



代码实现-主函数

```
int main()
{
    char filename[WORDLEN];
    FILE *fp;
    int pos, i;
    char word[WORDLEN];

    scanf("%s", filename);
    //打开文件, 如果失败, 则退出
    fp = fopen(filename, "r");
    if(fp==NULL){
        perror("Can't open the file!\n");
        return -1;
    }
}
```

```
//读取文件中的单词
while(getWord(fp, word)){
    //在单词表中查询单词
    pos = searchWord(word);
    if(pos >= 0){//找到了, 则次数加1
        wordTable[pos].count ++;
    }else{
        //没找到, 则将新单词插入单词表
        insertWord(word);
    }
}
//调用快速排序, 按照要求的顺序排序
qsort(wordTable, total, sizeof(WordRec), cmp);
//输出结果
for(i=0;i<total;i++)
    printf("%s %d\n", wordTable[i].word,
        wordTable[i].count);

return 0;
}
```



代码实现-头文件、全局变量和函数定义

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

//全局宏，定义单词长度和单词表长度
#define WORDLEN 55
#define TABLELEN 10005
//结构体，存储单词和数量
typedef struct _WordRec{
    char word[WORDLEN];
    int count;
}WordRec, *PWordRec;
//全局变量，单词表和单词数量
WordRec wordTable[TABLELEN];
int total = 0;
```

```
//从文件中读入一个有效的单词（小写形式），存入word;
//如无有效单词，则返回0，否则返回1
int getWord(FILE *fp, char word[]);
//查询单词表（全局变量）中是否存在单词word
//返回位置下标，如果没找到则返回-1
int searchWord(char word[]);
//在单词表（全局变量）的最后位置，插入单词word
//返回插入位置，如果插入失败，则返回-1
int insertWord(char word[]);
//快速排序用比较函数，先按次数从大到小，相同次数按字典序
int cmp(const void *p1, const void *p2);
```



小结

◆复习的知识点

- ✓字符串
- ✓数组、二（多）维数组
- ✓指针、指针数组、函数指针

◆新的知识点

- ✓了解字符串**KMP**算法原理
- ✓掌握**结构体**、**联合**的定义和使用
- ✓能够运用**结构体定义复杂数据结构**并完成各类操作
- ✓了解**文件**的基本原理，掌握文本文件的读写操作